Term Rewriting Systems

with Minimal Magic

Pum Walters

Copyright © 2022 Pum Walters
Published with Leanpub `https://leanpub.com/minimalmagicTRS`
See Also Minimal Magic blog (Term Rewriting)
Licensed under the Creative Commons Attribution-ShareAlike 4.0 International:

> Creative Commons Corporation ("Creative Commons") is not a law firm and does not provide legal services or legal advice. Distribution of Creative Commons public licenses does not create a lawyer-client or other relationship. Creative Commons makes its licenses and related information available on an "as-is" basis. Creative Commons gives no warranties regarding its licenses, any material licensed under their terms and conditions, or any related information. Creative Commons disclaims all liability for damages resulting from their use to the fullest extent possible

First Published, January 2022

Contents

1 Minimal Magic 7

2 Pum Walters 9

3 About this Book 11

I Term Rewriting

4 Minimal Magic 15

4.1 Magic 16

4.1.1 The Black Box 16

4.2 Minimal Magic 17

4.2.1 Interfaces 17

4.3 Summary 17

5 Term Rewriting 19

5.0.1 Sets of Turtles 19

5.0.2 Unbounded numbers of sets 19

5.1 Sets and Values 20

5.1.1 Representation 20

5.2 Sets as Larger Constructs 21

5.2.1 Pair, Tuple 21

5.2.2 Relation, Function 21

5.2.3 Symbols, Variables 21

5.2.4 Terms 21

5.2.5 Substitution, Instantiation, Matching . . . 22
5.2.6 Rules, Term Rewriting Systems . . . 22
5.3 Operationalization 23
5.3.1 Strategies . . . 23
5.4 Notation 25

6 Meta-Terms and Meta-Interpreter . . . 27
6.1 Rewrite Systems 27
6.2 Rewriting 28
6.3 Matching 28
6.4 Instantiation 29
6.5 Conclusion 29
6.5.1 TRAM.1 . . . 30

II Implementation

7 Tram: Term Rewriting Abstract Machine . . . 33
7.1 Design Criteria 33
7.1.1 Extensions . . . 34
7.2 Strategy 34
7.3 Memory Management 35
7.3.1 Nodes . . . 35
7.3.2 Values . . . 35
7.4 Stacks 35

8 Memory Management . . . 37
8.1 Mark and Sweep 38
8.2 Mark 38
8.2.1 Identify global objects (*global mark step*) . . . 38
8.2.2 Mark indirectly referenced objects (*recursive mark step*) . . . 39
8.3 Sweep 40
8.3.1 Fragmentation . . . 41
8.4 TRAM 41
8.4.1 Fixed-sized Nodes . . . 42
8.4.2 Persistence . . . 42
8.4.3 TRAM Garbage Collector . . . 42

9 Binary Numbers . . . 43

10 TRAM.1 . . . 47
10.1 Rationale 47
10.1.1 Tagged Values . . . 47
10.1.2 Memory . . . 48
10.1.3 Garbage Collector . . . 48

10.1.4 Scanner / Parser . . . 49
10.1.5 Printers . . . 49
10.1.6 Rewrite Engine . . . 49
10.1.7 CLI Argument Processor . . . 49

10.2 User Manual **50**

10.2.1 Tram.1 Syntax . . . 50
10.2.2 Tram.1 Semantics . . . 50
10.2.3 Formats . . . 51
10.2.4 Examples . . . 51
10.2.5 Distribution . . . 51
10.2.6 Installation . . . 51

11 Annotated TRAM.1 . . . 53

11.1 Basics **53**

11.1.1 Standard libraries . . . 53
11.1.2 Single Error Logger . . . 53
11.1.3 Tagged Value Type . . . 53
11.1.4 Predicates for Node, Variable, Symbol . . . 54
11.1.5 Conversion Pointer <==> Reference-value . . . 54
11.1.6 Node Structure . . . 54
11.1.7 Forward Declaration . . . 54
11.1.8 Stack Manipulation . . . 54
11.1.9 Declaration of Memory and Memory Management Variables . . . 54
11.1.10 Debugging Levels . . . 54
11.1.11 Counters for Logging . . . 55

11.2 Memory Management and Garbage Collector **55**

11.2.1 GC Registers . . . 55
11.2.2 Memory Initialisation . . . 55
11.2.3 Node Bit Manipulation . . . 55
11.2.4 Garbage Collection . . . 55
11.2.5 New Node . . . 56

11.3 I/O **57**

11.3.1 Forward Declaration . . . 57
11.3.2 Scanner / Parser . . . 57
11.3.3 Rewrite Engine States . . . 60
11.3.4 Conversion for Data Values . . . 60
11.3.5 Function Symbol Constants . . . 61
11.3.6 Identifier Decoding . . . 61
11.3.7 Optionally Print . . . 61
11.3.8 Print Value (Reference, Date, Variable or Symbol) . . . 61
11.3.9 Print Term . . . 61
11.3.10 Print TRS . . . 63

11.4 Rewrite Engine **63**

11.4.1 `rcat, rev` . . . 66

11.5 CLI Interpreter **66**

III How to Write TRS's

12 Converting C to Tram 73
12.0.1 Annotated Scanner / Parser in C 73
12.1 Scanner / Parser in Tram 78
12.1.1 `scaffix` 79
12.1.2 `sclini` 80
12.1.3 `scfinalize` and `scancollrls` 80
12.1.4 `scsym`, `scspec`, `scdec` and `schx` 80

13 Converting Tram To C 83
13.1 Preamble 83
13.1.1 States 83
13.1.2 Nodes 83
13.1.3 Basics 84
13.2 TRAM.1's Rewrite Engine 84
13.2.1 `burewr` 85
13.2.2 `toprewr` 86
13.2.3 `match` 87
13.2.4 `inst` 89
13.2.5 `ALLDONE` 90

14 Patterns: Encoding-Checker from Scratch 93
14.1 Pattern: Initializing Local Variables 93
14.2 Pattern: Function Name or Extra Parameter 94
14.3 Pattern: Have Your Cake 94

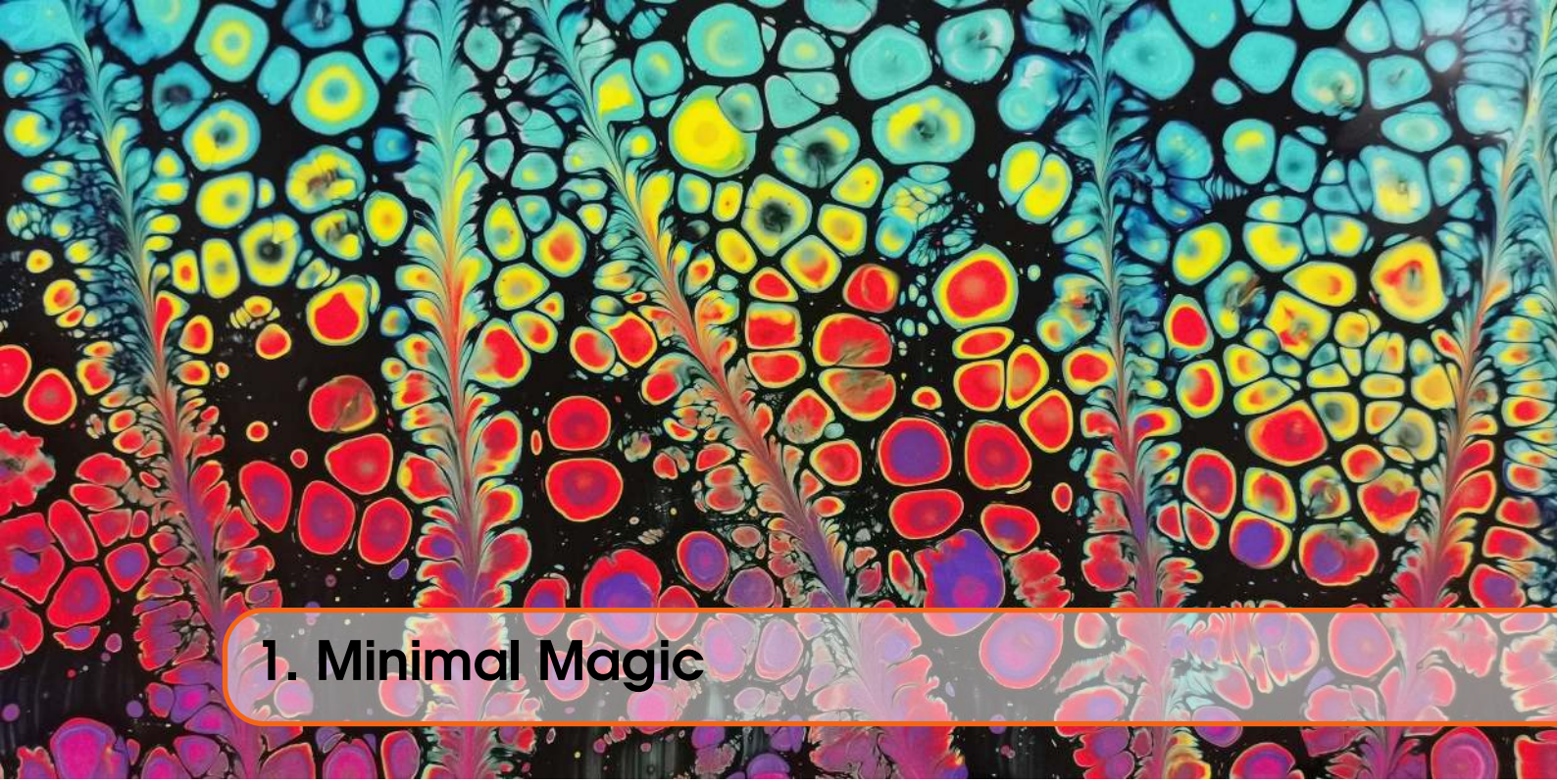
Conclusions

15 Conclusions 99

Postamble

16 Acknowledgements 103

17 References 105



1. Minimal Magic

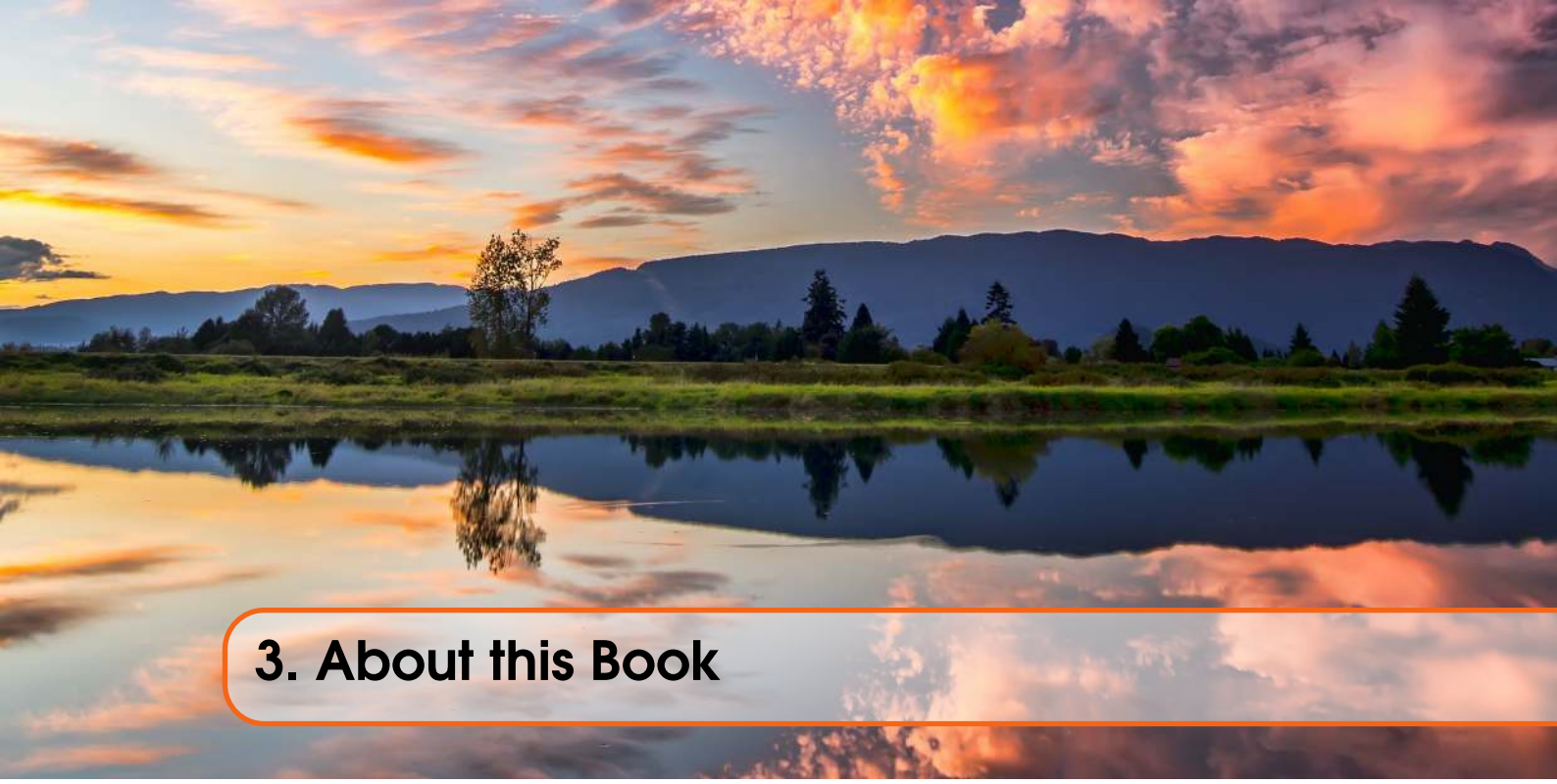
I once gave a freshman course on system engineering. The magic of system engineering is that once you understand a transistor, you can mostly understand an entire CPU. Later, I started writing a book on term rewriting with a similar foundation: *Ex Principiis Omnia*. This gave me the idea to start a blog with work and musings on that underlying theme.

2. Pum Walters

I am an independent researcher in Computer Science (Software Engineering, Cybersecurity, Enterprise Architecture, and Technical Computing). I have written various articles on Computer Science & Education and many parts of *as-yet* unpublished books and stories (both science and fiction).

Over the years, I've had many ideas for papers or books, and I only just now realized they often share a common theme, which is the reason to create this blog and which is expressed in the motto ***Ex Principiis Omnia***: everything follows from the beginning with **minimal magic**.

(Image: beautiful Iceland; that's not me)



3. About this Book

This book is a direct (automatic) rendition of the blog Minimal Magic, with only changes that affect the graphic design and leave further content as-is. Currently, internal links in the book refer to the blog. All photo's are taken (with permission) from the wonderful site Pexels or elsewhere when explicitly mentioned. All other images are (C) by Pum Walters.

Term Rewriting

4 Minimal Magic 15
4.1 Magic
4.2 Minimal Magic
4.3 Summary

5 Term Rewriting 19
5.1 Sets and Values
5.2 Sets as Larger Constructs
5.3 Operationalization
5.4 Notation

6 Meta-Terms and Meta-Interpreter 27
6.1 Rewrite Systems
6.2 Rewriting
6.3 Matching
6.4 Instantiation
6.5 Conclusion

4. Minimal Magic

Programming languages aim to balance performance and expressive power (among many other attributes) in the context of a consistent, intuitive programming model. Although these aims are no different for Tram, the focus differs from general programming languages:

- Performance:
  Tram is intended to test specifications and make prototypes. As such, practical performance is required, but if great speed is important, Tram may not be the ideal platform.
- Expressive Power:
  As this text illustrates, term rewriting systems are both austere and expressive: the underlying framework offers few features, but the language is highly extendable. While most languages support extension through class and function definition, term rewriting systems force one to think through the most appropriate control and data structure for a given problem. This trait is shared with other functional and logic languages. Worth mentioning are **Lisp macros**, which allow the definition of situation-specific language constructs, thereby turning Lisp into a **domain-specific language**;
- Programming model:
  Tram's programming model is part of its strength: terms and their equality may take some getting used to, but after that, there is no room for unclarity!

There is an apparent contradiction between high expressive power and the simplicity of the programming model. Expressive power requires the existence of powerful primitives, but the more primitives, or the more those primitives require deep understanding, the more complex the model becomes.

For instance, the Python documentation (version 3.9.1) introduces the list data type as follows:

> **Lists**
> Lists are mutable sequences, typically used to store collections of homogeneous items (where the precise degree of similarity will vary by application).

This description leans on a significant amount of tacit knowledge or at least knowledge that is explained elsewhere. For instance, 'sequences' are a built-in type that is defined in terms of an API that uses concepts of the underlying implementation language (macros, pointers).

Seemingly, to fully understand Python lists, one needs to dive deep into the implementation and into other languages. In fact, this is not the case. Most programmers are highly effective using a much less refined mental model and only turn to the nitty details of the specs when they observe unexpected behavior.

This has been the situation for almost all programming languages, and it is the way of the world.

By choice, in Tram, the strength and simplicity outweigh other concerns, which is formulated in the adage: 'minimal magic'.

4.1 Magic

'*Magic*' refers to built-in concepts that do not follow from the underlying theoretical framework, and '**minimal magic**' refers to the desire to minimize the level of concepts thus introduced.

Tram is based on set theory and therefore has the concepts of symbols, variables, terms, rules, and rewrite systems. One might say this mental structure is part of our minimal magic. Common prerequisites of programming languages such as data types, operations, data structures, control structures, modularity, etcetera are absent.

Unfortunately, one further bit of magic is inescapable, and in light of the above, it is fitting to characterize why.

4.1.1 The Black Box

Imagine a black box that implements a term rewriting system. In order to be of any practical or theoretical use, a user must have the ability to '*introduce*' a term in the unit and '*extract*' the normal form after it has been normalized. In addition, assuming the unit has finite memory, status information must be exchanged, such as '*out of memory*', when that situation occurs.

How could this introduction/extraction take place?

As we have seen, symbols are really sets of sets, and relations (functions, terms) are also sets of sets. How is a term outside the black box to be related to a set of sets (which may be represented as a number or not) in the black box? Without going into implementational details, one might state that a map and mechanism must exist between the external representation of terms and their internal representation.

It is reasonable to postulate that this necessitates one to identify at least two constants and a structuring element because:

- Information theory considers sequences of bits (0 and 1). This means that at least two constant and one structuring mechanism is defined;
- The Turing complete minimalistic language **Iota** considers sequences of two symbols: `i` and `*`, leading to the same conclusion;
- The Turing complete language **Jot**, closely related to Iota, considers possibly empty sequences of 0 and 1, again leading to the same conclusion.

Note that in all cases, two constants suffice and one structuring element. A structuring element is at least one function symbol (in the case of a binary operator).

Iota and Jot are full-fledged albeit minimal languages. It would be conceivable to compile term rewriting systems to Iota or Jot programs, and if minimalism were the only concern, it would be a very interesting approach. Clearly, just compiling to Iota or Jot isn't the whole story. One would then also have to implement that language, and one would still have to develop reversible mappings between symbols (identifiers) and Iota/Jot expressions.

Exchanging bits of information is another conceivable approach. One could imagine (for instance) data structures and control information encoded in bit patterns.

In any event, the interface between the term rewriting black box and its outside user must be unambiguous (and is considered magic because it doesn't follow from the theoretical term rewriting framework).

4.2 Minimal Magic

In the previous section, we have seen that the interface of the term rewriting black box should (at least) identify three symbols: two to *carry meaning* and one to *structure* the other two. But since most term rewriting systems define more than three symbols, it follows that aggregates must be used to identify symbols.

A naive approach suggests numbering all symbols in the term rewriting system and using those numbers (encoded in aggregates of the meaning-carrying constants) in the interface.

But there's a snag: the term rewriting system might not include all symbols that subject terms might contain. Consider, for example, a compiler implemented in a term rewriting system and assume variables in the subject language occur as symbols in terms representing programs. For instance, the statement `x=x+1` (in this imaginary language) might be represented as `let(x, plus(x,1))` (note that `x` is a variable in the language but obviously not in the term rewriting system). This observation is true for **every variable in the subject language**. But a compiler does not '*know*' all possible variables. The term rewriting system must be able to process terms containing symbols that are unknown beforehand. The only way to proceed is the ability to encode *all possible symbols*.

4.2.1 Interfaces

Using only three symbols may seem minimal, but if aggregates are then used to encode infinitely many further symbols that 'three symbol' minimalism becomes less meaningful. Not only the number of interface symbols should be considered, but also the encoding of other symbols. We will refer to these as **explicit** and **implicit** interface definitions. The explicit interface is the set of symbols used; the implicit interface is the set of symbols that is encoded in the explicit interface.

Tram uses the following **explicit interface** definition:

- The implementation defines 128 or 256 constants coinciding with the 7-bit ASCII characters or 8-bit bytes
- Implicitly, the sequence-operation strings characters or bytes together

Fewer symbols could be used. An alternative might have been to define only constants 0 and 1 and expect the implementation to process binary data. But (as discussed), that doesn't substantially 'minimize magic'; the choice to be able to process ASCII or binary files is practical and still minimal.

The **implicit interface** uses numbers to encode symbols and variables. The encoding differs per version of TRAM. TRAM.1, for instance, reversibly converts human-readable identifiers to 32-bit ints. As long as all occurrences of a symbol consistently use the same number to encode it, the semantics are sound.

4.3 Summary

To summarize,

- Tram is based on term rewriting, which is based on naive sets
- Tram uses an explicit interface of sequences of 7- or 8-bit data
- Each TRAM version uses an appropriate encoding as its implicit interface

5. Term Rewriting

Language definitions are often based on significant tacit knowledge. For example, the C *array* vs *struct* definitions refer to **contiguously allocated** vs **sequentially allocated** memory. Many C programmers would have trouble explaining that.

Our aim is to define an entire programming language without any prerequisites other than precision. No turtles required.

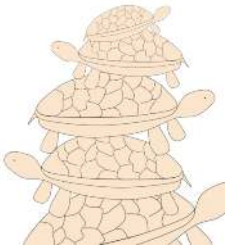


Figure 5.1: Turtles all the way down

5.0.1 Sets of Turtles

Consider these seven turtles. It is easy to imagine

- the set of right-facing turtles (blue box)
- the set of left-facing turtles (red box)
- the union (the entire set of seven)
- the intersection (empty)

Do we need turtles to talk about sets? No. We can define sets without turtles. Naive sets are sufficient (that is, *equality* if they have the same elements, *union*, *intersection*)

We have already seen one set without turtles: the empty set, written as `{}`. To be clear: there is almost no presumption here, just notation. All we presume is the empty set, not containing turtles or anything else. Let's give this set a name: `0`, so `0 = {}`

5.0.2 Unbounded numbers of sets

Now consider a set with exactly one element: the only thing we know so far, which is the empty set `{}`. This set is `{{}}`, and for clarity, we'll name it `1`. Clearly, `1` $\neq$ `0` because `1` has an element, and `0` doesn't. So now we now know two sets!

The next set we might consider is `{0,1}`, the set containing the two things we know so far, that are called `0` and `1`. We could also have written `{{},{{}}}` (this is why we give our sets names; braces are pretty soon unintelligible for humans).

This set `{0,1}` is again a new set because it has one more element than 1. And it reveals a pattern: given sets `0, ... ,N`, the set `{0, ... , N}` is a new set which we call `N+1`. Again: the use of names and numbers is a convenience – we do not presume the existence of numbers. As a matter of fact, using sets is a valid way to define numbers.

So, without significant presumptions, there is an unbounded number of unequal sets. We're avoiding the notion of infinity (which we don't need). In addition, we forego the precise definition of sets; for our purposes, the naive definition is sufficient. But in all fairness, we did make a presumption, which is: reasoning about finite sets.

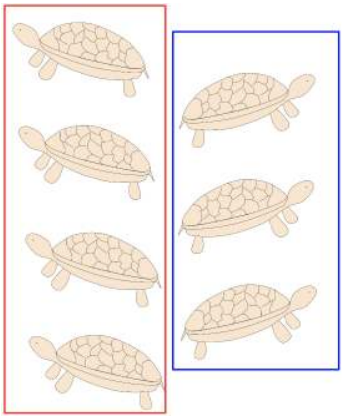


Figure 5.2: Seven Turtles

R

The **Zermelo Fraenkel** axioms are the most common basis of set theory and can be loosely phrased as

- Two sets are equal if they have the same elements
- Given two sets `x` and `y`, the collection `{x,y}` is also a set
- Given a set, the collection of its elements for which some property holds is also a set (this is called **comprehension**)
- The union of two sets is a set (as is the intersection, which follows from comprehension)
- The collection of all subsets of a set is also a set (and is called its **powerset**)
- The image of a set under a function is also a set

There are a few more Zermelo Fraenkel axioms, which are relevant for infinite sets and which are of no concern to us.

The notion of **functions** pops up out of nowhere. We will discuss this below. The last axiom is trivial for our finite sets.

5.1 Sets and Values

Starting from nothing, we have described mental artifacts called sets. We have given a sequence of these the names `0`, `1`, `2`, etcetera, so there is an unbounded number of them. Note that there are many more sets than the ones we have given names, such as, for instance: `{1, 3, 10}`. To distinguish the named sets from the unnamed sets, we will call the named sets: **values**. The unnamed sets will still play a role in our discourse.

It is easier for us to talk about values such as `3` than to talk about sets such as `{{},{{}},{{},{{}}}}`. But we haven't presumed numbers; we just reason about named sets.

5.1.1 Representation

As it happens, computers are rather good at storing and processing numbers represented as strings of bits in a format that is called **binary**. The relation between stored numbers and the real numbers (which are also a mental construct) is by convention. The view that `101` is related to the number 5 is just as sensible (albeit more common) as the view that it might be related to the set `5` (which is `{{{}},{{},{{}}},{{},{{}},{{},{{}}}}, {{{}},{{},{{}}},{{},{{}},{{},{{}}}}}}`).

Conceptually we are still within the theory of sets, but values (which are sets) can be represented as bits and bytes and binary numbers and can be processed by computers.

5.2 Sets as Larger Constructs

Now we will discuss several concepts based on the sets we have seen so far.

5.2.1 Pair, Tuple

Given two sets a and b the pair <a,b> is the set {a,{a,b}}. A pair is not a new concept but a notation. Note that from a pair, the constituent two elements can be unequivocally determined. A pair is a set of two elements, at least one of which is again a set with two elements. The original set and one of its elements have an element in common, which is the first constituent of the pair. The second element of the smaller set is also the second constituent of the pair.

If you are a programmer and unused to thinking about sets: a pair is somewhat like a C struct or JavaScript list.

Given N values v1, ..., vN a tuple <v1, ..., vN> is defined similarly to a pair. A tuple is a set from which the N constituents can be retrieved, so it is again similar to a struct or list.

5.2.2 Relation, Function

A **relation** is a set of pairs. Example: {<1,2>,<1,3>,<2,3>} is a relation. Note that we are mixing notations to remain intelligible: the braces {...}, brackets <...>, and numbers 1 notations. Just using braces, this example becomes:

{{{{}},{{{}},{{},{{}}}}},{{{}},{{{}},{{},{{}},{{},{{}}}}}},
{{{},{{}}},{{{},{{}}},{{},{{}},{{},{{}}}}}}}

Probably you are used to relations such as $\leq$ (less-than-or-equal). This is the same thing: {<1,1>,<1,2>,<1,3>,<2,2>,<2,3>,<3,3>} **is** the relation $\leq$ on the numbers 1, 2 and 3. As an aside, also note that this relation can also be defined as the 'is contained in' relation $\in$ because n<m (less than) if and only if n$\in$m (is contained in) considering n and m as names of sets.

A **function** is a relation in which each first value in the pairs is unique: the relation {<1,1>, <1,2>, <2,2>, ...} is not a function because 1 occurs twice as the first element in a tuple.

The notation f(x) and R(x,y)

The notation f(x): given a set of pairs f and a value x, f(x) denotes the value for which <x,f(x)> occurs in f. Similarly, R(x,y) means that <x,y>$\in$R.

A function f is a set of pairs, and f(x) selects a value contained in f. A programmer tends to think of f(x) as f being applied to or acting on x. There is nothing wrong with this view, but remember that underlying it is a statement about sets.

5.2.3 Symbols, Variables

Now consider two disjoint sets of values S={s1, ..., sN} and V={v1, ..., vM}. We'll call each si a **symbol** and each vi a **variable**. There are no restrictions on the sets of values and symbols. For example, one might choose the set of symbols to be all odd-named values and the set of variables all even-named values.

We give symbols and variables names (identifiers). By convention, an identifier starting with:

- an upper case letter is a variable: e.g. X, Bar
- a lowercase letter is a symbol: e.g. f, aX

Note that naming symbols and values is similar to middle-school practice for algebra and geometry. Consider the statement: x=1/2π. Here, x is the name of a variable, and π is the symbol used for the ratio between the diameter and the circumference of a circle.

Note: symbols and variables are values (sets) and can be represented as binary numbers.

5.2.4 Terms

A term is either a variable or a tuple <f,t1, ...,tk> (for k $\geq$ 1) where

- `f` is a symbol
- Each `ti` is a term

Confusingly, in our text, we also give names to terms, such as `t`. It should be clear from the context whether an identifier refers to a symbol or to a term.

The pair-notation isn't used for terms; we use the special notation `f(s1, ..., sk)`.

If `f` has no arguments, no parentheses are used: we write `f` instead of `f()` (again confusingly; but it should be clear from context whether we mean symbol `f` or term `f`).

Given a term `t=f(s1, ...,sk)`, the symbol `f` is called the **outermost function symbol** of `t`

Sub-term, Open / Closed Terms, Constant

Each `si` is a **sub-term** of `f(s1, ...,sk)`, and all sub-terms of each `si` are also sub-terms of `f(s1, ...,sk)`.

An **open term** is a term that contains variables; a **closed term** doesn't.

A closed term without sub-terms (e.g. `f`) is called a **constant** (somewhat confusingly, since any closed term is in a way a constant because it doesn't contain a variable, but there it is)

Trees

When terms are drawn, they appear tree-like as a structure of nodes and edges. In this sense, we often refer to the outermost level (the highest level or top) as the **root** and to the innermost (lowest) level as leaves.

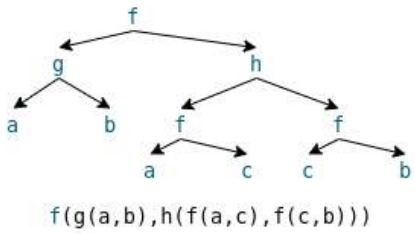


Figure 5.3: Term as a Tree

5.2.5 Substitution, Instantiation, Matching

A **substitution** is a function mapping variables to terms (i.e., a set of pairs `<vi, si>` where each `vi` is a unique variable, and each `si` is a term).

An open term `t` can be **instantiated** with a substitution `S` by replacing variables in `t` with their `S`-value.

For example: instantiating `s(a(X,Y))` with `{<X,s(z)>,<Y,s(z)>}` produces `s(a(s(z),s(z)))`.

Given a closed term such as `t=a(s(s(z)),s(z))` and an open term such as `p=a(s(X),Y)`, **matching** tries to find a substitution S such that S(p)=t. Matching can either fail or succeed in producing a substitution.

5.2.6 Rules, Term Rewriting Systems

A **rewrite rule** is a pair of terms written as `lhs → rhs` where

- `lhs` has at least one symbol (i.e., isn't a sole variable)
- every variable in `lhs` occurs at most once
- every variable in `rhs` also occurs in `lhs`

Given a rule, `lhs → rhs`, `lhs` and `rhs` are called the **left-hand** side and the **right-hand** side, respectively.

A term `t` can be **rewritten** to a term `u` by rule `lhs → rhs` if a substitution `S` exists such that `t=S(lhs)` and `u=S(rhs)`. Rewriting is a relation (called the **rewrite relation**), and by convention, we write it as `t → u` (confusingly, the notation for rules and for the rewrite relation are the same). That is: given a rewrite

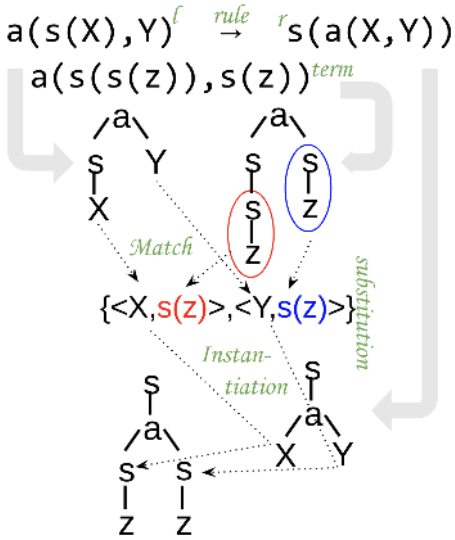


Figure 5.4: Matching, Substitution, Instantiation

rule, `lhs → rhs`, '→' is the rewrite relation between terms. In a more operational context, the rewrite-step relation is also called **reduction**.

This relation can be extended in an obvious way by rewriting sub-terms:
if `t → u` and if `t` is a sub-term of `v`, then `v → w` when `w` is obtained from `v` by replacing that sub-term `t` in `v` by `u`. A more precise definition follows in the last section of this page.

Secondly, the relation can be extended by regarding multiple rules at once: `t → u` if a rule `l → r` exists such that `t → u` for that rule.

A set of rewrite rules is called a **Term Rewriting System** (**TRS**).

To summarize, a term rewriting system defines a one-step rewrite relation (written as →) on terms, such that `t → u` means that

- a rule `l → r` exists in the TRS
- a substitution `S` exists
- `t` has a sub-term `p`
- `S(p)=l`
- `q=S(r)`
- `u` is obtained from `t` by replacing a sub-term `p` in `t` by `q`

The one-step rewrite relation can be transitively extended:
`s ↠ t` if, and only if `s → t` or `s → u` and `u ↠ t`. In other words: `s ↠ t` when a chained sequence of rewrite steps `s → u`, `u → v`, `v → w` ... `x → y`, `y → t`, exist.

If `s ↠ t` and no rule is applicable to any sub-term of `t` (or `t` itself), `t` is called a **normal form** of `s` (or **the** normal form if there is only one).

5.3 Operationalization

It might seem that this document is turning into an abstract theoretical discussion. So let's take a step back and see where we are and where we are headed.

Using only a naive sense of sets essentially founded only on the empty set, we have

- identified and named an unbounded number of sets, which we called values, and identified an even bigger collection of unnamed sets
- picked two disjoint collections, which we called symbols and variables
- sketched schemas to group sets in
  - pairs and tuples
  - relations and functions
  - terms, rules, and rewrite systems
- described the rewrite relation(s) on terms

This is all good and proper, but how can we implement it for practical use.

Before we do, we must make an operational choice.

To paraphrase 'rewrite step': given a TRS and a term, `s → t`

- if there is a rule `l → r`
- and if there is a sub-term `p` of `s` such that `p` matches `l`, then we can replace `p` with the corresponding instance of `r` to get `t`.

But what if more than one rule and sub-term exist for which this is the case? That is to say, what if given some term `s`, many terms `t` exist such that `s → t` or `s ↠ t`.

5.3.1 Strategies

R

Given some term `s`, many terms `t` might exist in general, such that `s → t`. The relation ↠ codifies whether one object can be derived from another object by repeatedly applying →. So given a term `s`, many paths might lead to many terms

t such that s ↠ t. Indeed, some of those paths might even be infinite. For instance, the rule a → i(a) leads to infinite reductions to ever-growing terms.

Two important properties of term rewriting systems are:

- **Confluence**
  if for any w, x, y such that w ↠ x and w ↠ y, then a z exists such that x ↠ z and y ↠ z
- **Termination**
  there are no infinite reduction chains u → v → w → ...

These properties are important because confluence guarantees that even when diverging reductions are possible, we can always converge, and termination guarantees there are no unending paths. Together they guarantee that **every term has a unique normal form**.

However, it is generally undecidable if a TRS is terminating and/or confluent and significant research exists to determine if term rewriting systems have these desirable properties or under which (syntactical) restriction the properties can be proved.

A more operational view is that confluence and termination are similar to non-determinism and infinite loops in software: undesirable circumstances where it is up to the programmer to avoid them.

Strategies are important because they are strongly related to these important properties of confluence and termination: A term rewriting system that is not confluent and/or terminating may be confluent and/or terminating when limited to a specific strategy.

For example, the term rewriting system below is not terminating, but if one rigorously applies the third rule before any other rule, every reduction sequence terminates.

```
f(a) → f(b)
f(b) → f(a)
a → c
```

A **rewrite strategy** determines which rule and which sub-term to pick.

Sub-Term Selection

The *innermost* and *outermost* strategies limit the location where reductions are considered. An **innermost strategy** reduces a sub-term only if all deeper sub-terms of that sub-term can not be reduced. An **outermost strategy** reduces only sub-terms of a term if they aren't contained in a (sub-) term which is reducible at the root level.

Consider this TRS:

```
f(a) → f(b)
f(b) → f(a)
a → c
```

This TRS is terminating under an innermost strategy but not under an outermost strategy.

Now consider a function `if` for which two rules are given:

```
if(true,X,Y) → X
if(false,X,Y) → Y
```

Consider a term `if(B,T,E)` where `B`, `T` and `E` are given terms for a boolean expression, the 'then' part and the 'else' part. Clearly, any reduction in the *then* or *else* parts may prove to be irrelevant until the Boolean expression is reduced to either `true` or `false`.

If `B` proves to be `true`, any reductions in the *else* part were pointless (when one considers an implementation, superfluous reductions represent wasted time) and potentially *harmful* if the *else* part leads to infinite reduction sequences (which may not be a bug if `B` is `true`.

A safer and more optimal strategy (for instance, the left-most outermost reduction strategy) evaluates the Boolean, and only then reduces *if*.

Speaking generally, outermost strategies are 'better behaved' with respect to termination, but innermost strategies can be implemented more efficiently. While it is true that innermost strategies can lead to non-termination, it is generally straightforward to write a term rewriting system such that non-termination is avoided in the same way that infinite loops should and can be avoided in general programming languages.

Rule Selection

A second aspect the strategy must address is: if more than one rule is applicable at the same location, which is chosen? If the term rewriting system is confluent, it doesn't matter in a sense. But it may be unknown if the term rewriting system is, in fact, confluent if non-termination lurks, or, in an implementation, if one choice might be more efficient than another.

Common strategies which address this are *specificity order* and *textual order*.

- **textual order** always applies the first rule (in the textual representation of the TRS) that is applicable (to the sub-term under consideration);
- **specificity order** always applies a rule which is at least as specific as all other applicable rules. Specificity can be defined as the total number of function symbols on the left-hand side. Specificity order makes intuitive sense because otherwise, that more specific rule would never be considered!

As an example, consider a function `iszero`, which tests if a number in the *a-s-z-system* (see also at the end of this section) is zero.

```
iszero(N) → false
iszero(z) → true
```

Using textual order, this term rewriting system always returns false, but using specificity order, the implementation behaves appropriately.

To be honest, this TRS is unnecessarily confusing, especially for a programmer who is used to reading code from top to bottom. *By convention, the order of rules is always given such that textual order and specificity order coincide* as much as possible.

```
iszero(z) → true
iszero(N) → false
```

R

Some reduction strategies address location selection and rule selection in one go. *Priority* and *annotation* strategies combine the selection of the location and of the rule being applied.

A priority strategy assigns priorities to symbols and only allows a reduction anywhere if no higher priority symbol can be reduced. Alternatively, priorities can be assigned to rules with similar results. In an annotation strategy, some or all rules are annotated with information on the order of (considered) reductions.

For example, the if-true-false system shown earlier might indicate that for an *if*-symbol, the first argument must be normalized before the other arguments are considered.

5.4 Notation

So far, we have used this notation `l → r` for rules.

```
iszero(z) → true
iszero(N) → false
```

But the symbol → doesn't occur on any keyboard, and the lack of a separator between rules might lead to difficult-to-read code.

We will use the symbol = instead of → in rules and terminate all rules with `;`.

In this document, rule numbers are often included for reference. They are not part of the spec but only of the presentation.

The above TRS would then appear as:

```
iszero(z) = true;
iszero(N) = false;
```

The use of = (equality) deserves a comment: rewrite rules and rewriting are directional, always being applied from left to right. On the other hand, they do express equality. The predicate `iszero` applied to `z` is rewritten to `true` precisely because `iszero(z)` **is (equal to) true**. Rewriting is *syntactical* simplification within *semantical* equality.

Example

Consider the following TRS ('*the a-s-z-system*'; successor-zero with addition):

```
a(s(X),Y) = s(a(X,Y));
a(z,X) = X;
```

and consider the following sequence of rewrite steps:

```
a(s(s(z)),s(s(z)))
→ s(a(s(z),s(s(z)))
→ s(s(a(z,s(s(z)))))
→ s(s(s(s(z))))
```

Now, interpret

- `z` as zero
- `s` as successor (plus one)
- `a` as add

We have just defined addition for unsigned integers!

6. Meta-Terms and Meta-Interpreter

Earlier, we saw that a term is either a variable or a tuple `<f,t1, ...,tk>` (for `k` $\geq$ `1`) where `f` is a symbol and each `ti` is a term.

When describing terms (and term rewriting systems) **within** a term rewriting system, it is necessary to have a meta-representation of terms. In the meta-representation:

- A term is represented as
  - `trm(F,Args)` for symbol `F` and terms `Args`, or
  - `var(N)` for variable named `N`
- Arguments `Args` are
  - `arg(Term, Args)` for some Term
  - `eoa` end-of-arguments, the empty list
- For now, we ignore the inner structure of function symbols or variables in this meta-interpreter. We merely state that a function `eq` exists, which returns `ok` when applied to equal function symbols or variables.

For example, the term

```
add(succ(succ(zero)),succ(zero))
```

is represented by:

```
trm(add, arg(trm(succ, arg(trm(succ, arg(trm(zero, eoa),
   eoa), eoa), eoa)), arg(trm(succ, arg(trm(zero, eoa), eoa),
   eoa), eoa)))
```

6.1 Rewrite Systems

A term rewriting system (i.e., a list of rules) is represented as

- `rl(Lhs,Rhs,Rules)` for terms `Lhs` and `Rhs`
- `eor` (end-of-rules)

Note that in many places, the meta-representation of terms is used for rules. For instance, the Scanner / Parser produces a meta-representation of a term rewriting system. That is, `rl`

and `eor` are considered (pre-) defined function symbols but have no special significance in the meta-representation of terms; the meta-representation of a TRS is that of a term.

```
trm("rl",
  arg(trm(... Lhs ...),
  arg(trm(... Rhs ...),
  arg(trm(...Rules...),
  eor
)
```

In this section, we present a TRS which implements term rewriting using an innermost strategy (rules 1&2, below) interpreting rules in textual order (rule 13).

6.2 Rewriting

- Rule 1 states: after all sub-terms have been reduced, try to reduce at the root level.
- Rules 2 and 3 attempt to reduce innermost sub-terms.
- Rule 4 attempts each rule at the top level. A copy of the list of rules is kept because there are no global variables. Once a reduction takes place, the entire TRS must be applied again to the reduced term.
- Rule 5 attempts to match (which will instantiate if a match is found)
- Rule 6 terminates the entire rewrite process: the term is a normal form.

```
1 burewr(trm(F,As),Rs) = toprewr(trm(F,burewr(As,Rs)),Rs);
2 burewr(arg(A,As),Rs) = arg(burewr(A,Rs),burewr(As,Rs));
3 burewr(eoa,Rs) = eoa;

4 toprewr(T,Rs) = toprewrRule(T,Rs,Rs);

5 toprewrRule(T,trm("rl",arg(L,arg(R,arg(Rs,eoa)))),TRS)
    = match(L,T,eol,eoa,eoa,R,T,Rs,TRS);
6 toprewrRule(T,trm("eor",eoa),Rs) = T;
```

6.3 Matching

Function `match` has nine arguments:

- pattern being matched,
- term being matched,
- list of variables seen so far
- remainder of the pattern to be matched
- remainder of the term to be matched
- right-hand-side of the rule being considered
- entire term (needed if this match fails)
- remaining list of rules (needed if this match fails)
- entire TRS, needed if a match succeeds, and we need to continue after this rewrite step

The rules are straightforward:

- Rule 7 adds a variable-value pair to the list of variables seen so far. Note that this rule assumes that every variable occurs at most once on the left-hand side

- Rule 8 checks if the corresponding function symbols are equal
- Rule 9 attempts to match sub-terms
- When one subterm (and sub-pattern) has been successfully matched, the list of remaining sub-terms (and sub-patterns) are considered
  - (10) if there are none, matching has succeeded, and instantiation should take place
  - (11) otherwise, matching continues
- Auxiliary function `matchq` either (12) continues matching (if sub-term and pattern have the same corresponding function symbol), or (13) attempts to apply the remainder list of rules if the match failed.

```
7 match(var(N),ST,E,As,Bs,R,T,Rs,TRS)
    = match(As,Bs,tab(N,ST,E),eoa,eoa,R,T,Rs,TRS);
8 match(trm(F,Fs),trm(G,Gs),E,As,Bs,R,T,Rs,TRS)
    = matchq(eq(F,G),Fs,Gs,E,As,Bs,R,T,Rs,TRS);
9 match(arg(P,Ps),arg(Q,Qs),E,As,Bs,R,T,Rs,TRS)
    = match(P,Q,E,arg(Ps,As),arg(Qs,Bs),R,T,Rs,TRS);
10 match(eoa,eoa,E,eoa,eoa,R,T,Rs,TRS) =  inst(R,E,TRS);
11 match(eoa,eoa,E,arg(A,As),arg(B,Bs),R,T,Rs,TRS)
    = match(A,B,E,As,Bs,R,T,Rs,TRS);

12 matchq(true,Fs,Gs,E,As,Bs,R,T,Rs,TRS)
    = match(Fs,Gs,E,As,Bs,R,T,Rs,TRS);
13 matchq(X,Fs,Gs,E,As,Bs,R,T,Rs,TRS) = toprewrRule(T,Rs,TRS);
```

6.4 Instantiation

Instantiation is straightforward:

- (14) a variable is instantiated with the value with which it was matched. Note that every variable on the right-hand side must have been matched, so there is no check for the reverse
- (15-17) all compound terms are recursively instantiated
- (18-20) simple table lookup of a matched variable

```
14 inst(var(N),E,TRS) = get(N,E);
15 inst(trm(F,As),E,TRS) = toprewr(trm(F,inst(As,E,TRS)),TRS);
16 inst(eoa,E,TRS) = eoa;
17 inst(arg(A,As),E,TRS) = arg(inst(A,E,TRS),inst(As,E,TRS));

18 get(N,tab(M,T,E)) = getq(eq(N,M),N,T,E);

19 getq(ok,N,T,E) = T;
20 getq(X,N,T,E) = get(N,E);
```

6.5 Conclusion

The upside of a meta-interpreter is that it offers a precise (correct, complete, clear, and concise) definition of innermost term rewriting. So far, we have been informal, sketching rather than formally detailing term rewriting, but the meta-interpreter changes that.

6.5.1 TRAM.1

A downside of a meta-interpreter is that it doesn't directly bring us closer to an implementation – it only offers an implementation if one already has an implementation.

TRAM.1 is an implementation written in C, which nonetheless follows the structure of the meta-interpreter. The key difference is that the meta-interpreter uses recursion heavily, while recursion in C is usually limited to tens of thousands of levels. That may sound like a lot but consider: `match` is attempted recursively through all rules (Rule 13), and when a match is found, `inst` is called (Rule 10), which recursively continues reduction (Rule 15). This means the depth of the recursion is the length of the rewrite sequence, which can be anything. This recursion must be transformed to iteration in C.

TRAM.1 is presented further in TRAM.1

II Implementation

7 Tram: Term Rewriting Abstract Machine 33
7.1 Design Criteria
7.2 Strategy
7.3 Memory Management
7.4 Stacks

8 Memory Management 37
8.1 Mark and Sweep
8.2 Mark
8.3 Sweep
8.4 TRAM

9 Binary Numbers 43

10 TRAM.1 47
10.1 Rationale
10.2 User Manual

11 Annotated TRAM.1 53
11.1 Basics
11.2 Memory Management and Garbage Collector
11.3 I/O
11.4 Rewrite Engine
11.5 CLI Interpreter

7. Tram: Term Rewriting Abstract Machine

TRAM (Term Rewriting Abstract Machine) is an implementation of term rewriting systems. There are a few versions named TRAM.1, etcetera, in which different design choices have been made.

We use the notation **TRAM** (all caps) to refer to the implementation, the abstract machine, or the rewrite engine, and we use the notation **Tram** (lowercase) to refer to the input language of such systems.

Tram supports left-linear unconditional term rewriting systems. TRAM is created in the spirit of

- [Walters & Kamperman, 1993]
- [Walters & Kamperman, 1996]
- [Fokkink et al., 1998]

Tram is available at Github

7.1 Design Criteria

In Section Term Rewriting, we discussed criteria that make term rewriting a suitable basis for general programming and specification. Term rewriting systems offer a minimalistic, powerful, unopinionated framework for the design, specification, and implementation of software:

- **Unopinionated**
  There are no built-in numbers, strings, arrays, objects, etcetera. This is bad if you want a flying start; use Python in this case. It is good if you want to be precise and use TRSs both as a specification and an implementation language.
- **Minimalistic**
  In this section we discuss **minimal magic**: term rewriting systems can be explained without any significant assumptions or tacit knowledge.

- **Powerful**
  Powerful is a subjective and contextual term. Here we take it to mean "*capable of specifying or implementing complex structures and mechanisms*".

The design criteria for TRAM follow:

- **Small and easy to understand in minutes**
  TRAM.1 is small (~650 lines of standard C code using a few standard libraries. One reason to make TRAM light-weight is the underlying goal to implement TRAM directly on FPGA hardware
- **Reasonably efficient**
  TRAM is reasonably efficient (e.g., significantly more efficient than a naive recursive implementation).
  - The absence of assignment in pure term rewriting means that the language is side-effect free, and terms are never altered. Consequently, a single-pass garbage collector suffices
  - TRAM represents terms using nodes with fixed arity (2). Consequently, a non-compacting garbage collector suffices
  - TRAM can use a so-called 'bytecode engine', assigning part of the work to the compiler. This approach is foreseen in TRAM.3
  - TRAM uses an innermost reduction strategy which means that only terms are built which are in normal form
- **Capable and self-contained** TRAM.1 is a stand-alone tool suitable for batch processing

Notes:

- At the time of writing, TRAM is implemented using 32-bit words.
- Many conceivable extensions might make TRAM more practical, such as integers, floats, external functions, etcetera. But TRAM is implemented from the tenet that TRSs offer an expressive general-purpose framework that, with the exception of high-speed or memory-intensive applications, is truly sufficient. Accordingly, TRAM is created with a very low degree of magic.

7.1.1 Extensions

Several extensions/adaptations can be named which may be added to TRAM at some point, such as

- direct memory stacks instead of stacks-as-terms
- indexes to speed up execution

Tram is intended to be minimalistic, and many extensions can be added as source-to-source transformations. For example:

- Nonlinear left-hand sides
- Other conditional rewrite rules

A few features could have been handled this way but are deemed crucial to the development effort:

- Tram has comments in order to make source code self-documented
- In Tram, rewrite rules are written as `lhs = rhs;`. Strictly speaking, the meta-notation of terms defines the symbols `rl` and `eor` to represent lists of term (`rl(lhs1,rhs1, rl(lhs2, rhs2, ..., eos)..)`). However, that meta-notation is difficult if not impossible to maintain by humans. Also, the implementation uses a more compact and more efficient model to store rules.

7.2 Strategy

Before diving into TRAM, we first set the stage.

A term rewriting system (TRS) is an ordered set of rewrite rules. Given a *strategy* and a *subject term*, term rewriting is an iterative process in which a sub-term of the subject-term and a rule are identified, such that the left-hand side of the rule matches that sub-term. Then, that sub-term is replaced by the corresponding instance of the rule's right-hand side. This iteration is repeated until no more reducible sub-terms can be found and the resulting subject term is in normal form. If the

TRS is confluent and terminating, this process leads to a unique normal form; otherwise, the result may be non-deterministic or the iteration may never terminate, which may be a bug but might also have sensible applications.

As described in Section Term Rewriting, we limit ourselves to two aspects of strategy: choosing a sub-term and choosing a rule to be considered. Two options were mentioned for each choice: innermost and outermost strategies, and textual order, and/or specificity order.

Outermost strategies are somewhat better behaved with respect to termination, but there is an overhead: terms are built which are not normal forms. That is a waste of resources. As we will see, in an innermost strategy, only normal forms are built; all other function symbols remain on a stack (which has less overhead in principle). For the moment, we consider innermost reduction only.

Secondly, textual order can be applied such that it coincides mostly (though not always) with specificity order. In addition, textual order coincides with common programmers' experience. For the moment, we consider textual order (though we will revisit this choice).

7.3 Memory Management

Section Memory Management offers an introduction and rationale of TRAM's memory management and garbage collection. Section Term Rewriting offers an introduction and theoretical foundation of the concepts of symbols, variables, terms, rules, matching, instantiation, etcetera.

TRAM distinguishes three data types: symbols, variables, and compound terms. Symbols and variables are represented as numbers (32-bit ints in TRAM.1).

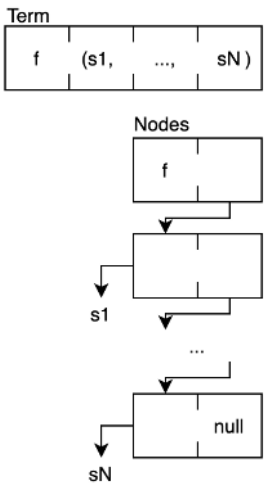


Figure 7.1: N-ary term represented using binary nodes

7.3.1 Nodes

TRAM terms are represented with nodes. A node is a structure with three fields called `car`, `cdr`, and `nxt`. `Nxt` is used for memory management and will be discussed elsewhere. The fields `car` and `cdr` hold a value, which is a symbol, a variable, or a reference to another node.

The transformation of terms with arbitrary arity (number of arguments) to binary tree structures happens in TRAM parsers. TRAM engines are only concerned with binary nodes.

An entire term is represented as a set of linked C structs. The term is represented as a value by storing a reference to the root struct of that term. This reference is direct (TRAM doesn't need or use handles). Since TRAM.1 uses 32-bit numbers, a reference can't be a pointer, so an offset is stored. **A reference to a term is a 32-bit index in the array of all nodes.**

7.3.2 Values

To summarize, values are 32-bit ints which represent

- symbols
- variables
- nodes (a reference to the root node of a term)

7.4 Stacks

A term rewriting implementation uses stacks for

- The parser. Terms use parentheses, which implies that a scanner is insufficient and a parser is required, which must use a stack. Strictly speaking, push-down automata could also be used. At this point in time, that idea hasn't yet been applied.
- Intermediate results. As mentioned, an innermost strategy refrains from building terms until they are known to be normal forms. But the outermost function symbols that have yet to be matched must be stored somewhere, and (as mentioned) a stack is appropriate.
- Recursion. As mentioned, the highly recursive nature of the meta-interpreter has been implemented with iteration, but that iterative process still needs storage, which is stack-like.
- Finally, to produce output, the engine needs a stack while printing terms.

To simplify TRAM, these stacks can be implemented using TRAM's own memory manager (i.e., nodes). *This approach is simple but inefficient.*

8. Memory Management

Software uses memory to store data objects. An object may contain data such as strings or numbers, but it may also contain references to other nodes. The collection of all nodes is called a **heap**.

The lifetime of an object is from the moment it is created to the first moment after which it will never be accessed. After its lifetime, the memory an object occupies could be reclaimed for reuse.

When software isn't overly complex, an object's end of life follows from the logic. At that time, a handler can be called to reclaim the memory. This is called **Explicit** memory management. For instance, most operating systems use this approach with the simple C API `malloc`, `realloc`, `calloc`, and `free`.

But in more complex situations, it is impossible to determine the moment after which an object is no longer needed (the question of whether a program will use some object in the future is undecidable in general).

Automatic memory managers use conservative approximations of the end-of-life: some moment at which it is certain the object will not be accessed anymore. Many approaches are based on the observation that *if a program no longer has access to an object, it is safe to reclaim it*.

Actually, 'automatic' is a misnomer: software libraries are used to govern access and administer state, hiding management from the programmer; it's only automatic from the programmer's perspective. This is impossible in general but effective in specific cases such as abstract machines (which follow predictable engineered patterns) or circumstances where the programmer can be required to follow such patterns.

One example of memory management is **reference counting**: a hidden field is maintained in every object, which counts the number of references to that object. Once the counter hits zero, the memory can be reclaimed. This approach is useful and efficient when:

- the moment an additional reference is created or deleted can be easily captured;
- data structure cycles are prevented or detected;
- the additional memory used by the hidden field, and the computational overhead, are small compared to the user-task.

However, this approach only makes sense in a relatively small number of cases.

8.1 Mark and Sweep

Most memory managers use accessibility criteria in a two-phased approach called **Mark and Sweep**:

- in the mark-phase, all objects accessible to a program are marked
- in the sweep-phase all unmarked objects are reclaimed

This approach is complex in general (we will describe issues below), but under certain circumstances (i.e., restrictions), that complexity can be reduced. Two criteria to look at are:

- storage space: most systems require some additional storage for bookkeeping. For example, the reference counter mentioned above requires an integer field in every object;
- execution time: the mark and sweep algorithm takes time and (depending on the details) may need to block the operation of the user program while it is executing.

Before describing issues and restrictions which mediate those issues, let's discuss common overhead.

- **time overhead**
  There is no absolute acceptable time-overhead for memory management, including garbage collection. Many systems perform at about 3-5% (i.e., 3-5% of overall execution time is spent in garbage collection). In addition, if garbage collection blocks the user program, an absolute maximum of 0.5 seconds or so is accepted in interactive programs. Note that memory management often entails some bookkeeping overhead in addition to garbage collection itself, which is difficult to measure.
- **space overhead**
  There is no absolute acceptable space-overhead for memory management, including garbage collection. Many systems perform at about 30-50% overhead (i.e., 23-33% of overall space is allocated to implement memory management).
- Space and time overhead cannot be separated. If speed is an issue, more storage can be allocated for a faster algorithm and vice versa. Additionally, as memory in use approaches total available memory, many approaches deteriorate in efficiency. Quantifying the Performance of Garbage Collection vs. Explicit Memory Management, Herz & Berger shows (among other things) that a memory overhead of 500% can reduce the time overhead to almost 0.
- acceptable time and space overhead is also influenced by the average size of nodes and the frequency with which they are allocated and reclaimed. Some languages use nodes essentially in their execution model (e.g., functional languages), whereas others use them only to build persistent data structures. In any event, if small nodes are used frequently, space and time overhead must be kept to a minimum.

8.2 Mark

The mark phase generally consists of two steps:

- Mark all directly accessible objects (*global mark step*)
- For every marked object that contains references to other objects, mark those (*recursive mark step*)

Note that *'directly accessible objects'* includes objects in (referenced by) global variables, but also those in local variables stored in stack frames and possibly other places.

We will discuss several issues.

8.2.1 Identify global objects (*global mark step*)

Can global objects be identified by garbage collector code that is unaware of the logic of the user program? In general, this is impossible. The garbage collector is activated when a new node is to

be created, and no memory is available, but other than that, no information about the process state is available.

- Which global variables contain (a reference to) objects?
- How is the stack structured in which local variables can be found?
- Which CPU registers contain such references?
- How are other intermediate results stored?

Approaches to solving this can be readily imagined.

Compilers, Libraries

If the compiler being used is aware of memory management and garbage collection, it can handle access to objects (for instance, based on type information) in a special manner.

All object creation and access are compiled with additional code using a library to implement memory management and safeguard consistency.

This situation is most appropriate in languages that include memory management, such as the **go** programming language. This situation may introduce some space and time overhead but is generally regarded as the most efficient approach. However, it is only suitable in specific cases.

Using this approach doesn't say how memory management is technically implemented. It can be based on reference counting or on any of the methods discussed below.

Handles

Instead of direct pointers to objects, pointers to handles could be used. A **handle** is a small object which contains only a pointer to the actual underlying object. Handles can be made known to the garbage collector (using a subscription mechanism or memory layout). Handles can also play a role in the sweep phase, as we will see.

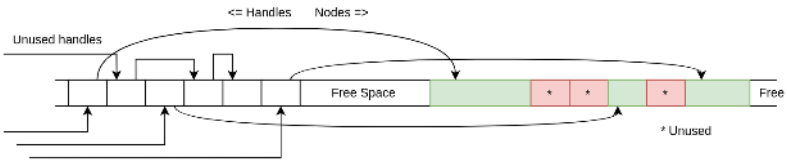


Figure 8.1: Handles

This solution introduces overhead both in

- **time**
  An additional memory access is needed almost for every object access. In addition, no intermediate pointers within the objects are allowed. For instance, if the object is an array, the common C practice to process all elements using a pointer within the array would be complicated with additional control mechanisms to safeguard the integrity, such as a locking mechanism.
- **space**
  One additional pointer per object would be acceptable in most memory management implementations.

If the client program is an abstract machine such as a bytecode interpreter, its entire state can be known, and handles could be used.

Note that the time overhead of using handles can be very significant. Handles are most commonly used for larger objects or where some form of locking isn't prohibitive.

Also, note that not only the space used by proper objects should be garbage-collected, but also the space used by handles. However, this can be very efficiently done (see Section 'Sweep' below)

8.2.2 Mark indirectly referenced objects (*recursive mark step*)

How can indirectly referenced objects be identified?

A naive description is a recursive algorithm:

```
(1) for each marked object o
(2)     mark children of o
```

There is quite some complexity hidden here:

How to visit all objects

If objects are laid out adjacently in memory, one might visit all objects by walking linearly through memory. But given an object in memory, it is not necessarily possible to find the next object in use in memory. It might be at the next word boundary or some other boundary. But if the next fragment of memory is unused, it may not be formatted as an object, and it is impossible (in general) to find the next used object. At the price of some overhead, this could be solved.

But even if line (2) in the algorithm introduces a problem because it extends the set of '*used*' objects. An object which seems unused at the first sequential pass may turn out to be in use after all. Multiple passes over the entire memory would lead to too much time overhead.

Another way to look at the algorithm is as a recursive function. But the worse case behavior of such a function is to recurse across all objects in memory (cycles are not a problem because we recurse only on unmarked objects). The problem is that the garbage collector is called only when memory is scarce, so there may not be enough stack memory to implement this recursive function.

Queues offer an alternative mechanism:

- in step one, all directly accessible objects are queued
- repeatedly, an object is dequeued, flagged, and all its unflagged children are queued
- this is repeated until the queue is empty

The worst-case behavior of this implementation is also to queue all objects, but that is limited and could be implemented using one additional pointer per node and no additional time overhead other than garbage collection. This approach is used fairly often.

Link Inversion

Heap traversal when memory is scarce is an area of research by itself. Over the years, many approaches have been developed. One such approach uses link inversion: when one traverses a tree, two pointers keep track of one parent and one child, and the pointer in the parent to the child is inverted to point to the parent's parent (the inverted links form a stack of parents). No extra memory is needed, and no information is lost. An index in the parent may be needed to identify which of its child-pointers has been inverted.

Conclusion

No ultimate conclusion is reached: mark and sweep are somewhat intertwined, so a holistic approach must be taken.

8.3 Sweep

Two situations must be distinguished:

- nodes have equal size
- nodes have variable size

If nodes have equal size and if that size is big enough to hold bookkeeping overhead (e.g., a pointer), the world is simple: after the mark phase, all unused nodes are linked in a list to be reused. If nodes have equal size, but that size is too small for linking, a bitmap can be set aside to identify unused nodes.

8.3.1 Fragmentation

If nodes have different sizes, fragmentation lurks: memory consists of many differently sized, used nodes interspersed with unused nodes and other unused fragments. There are two challenges:

- how to combine adjacent unused fragments into bigger (unused) chunks?
- how to combine unused chunks which are separated by used nodes?

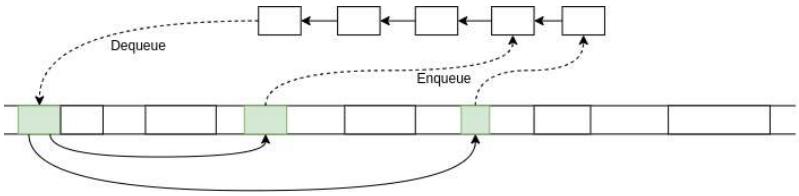


Figure 8.2: Queue

Combining adjacent unused fragments may seem simple, but if no special care is taken, it involves looping through the entire memory: a significant time overhead. Even if unused nodes are kept in a linked list, finding adjacent ones is problematic: it may involve ordering that list, which is time overhead. But: systems exist which employ this approach (e.g. buddy memory allocation). Especially if sorting is only local, the overhead can be limited.

Unused chunks separated by used nodes are the tougher problem. If they are not joined, a request for a big node might be unfulfillable even when enough unused memory is available (albeit fragmented). In time, performance would degrade significantly.

The most common solution is **compactification**: moving used nodes together to end up with one (or a few) unused chunks. Compactification may seem like an extravagant overhead, but it is worthwhile in many relevant cases. And the overhead can be limited using several techniques:

- In **generational** approaches, the lifetime of used nodes is noted. If a node has survived (i.e., continues to be in use) after many garbage collection cycles, it is unnecessary to be considered for compactification every cycle. It is put together with other older-generation nodes, which are compactified only occasionally.
- **Size pools** are used if nodes have one of a few sizes (or if all nodes fit within such a container). In this case, it makes sense to pool all equally-sized nodes/containers together, using techniques for memory management of equal-sized nodes. Obviously, This introduces the concern of disproportionally grown pools, which must then be solved.
- **Roving** refers to a technique to analyze or execute garbage collection functionality in parallel to user program execution. That way, when the garbage collector is called, (partial) compactification can be executed immediately without time overhead. Note that overall overhead isn't reduced, only overhead for the garbage collector itself.

The biggest concern of compactification is the question of whether used nodes can easily be moved. If a node is moved, all references to (and within) that node must be updated. This means all global and local variables and all other places where an intermediate result might point to/in an object (e.g., some CPU register) should be updated.

In general, this is all but impossible. In specific situations such as abstract machines, this may be possible. A common solution is the use of **handles** because then only the handle needs to be updated. But, as mentioned, this is only suitable in some situations and does require every node access to go through the handle every time, which constitutes significant overhead in general.

8.4 TRAM

The Term Rewriting Abstract Machine (TRAM) implements memory management as follows.

8.4.1 Fixed-sized Nodes

Terms can have any arity (immediate sub-terms) and would therefore require variable-sized nodes. This requires compactification, which would lead to significant overhead.

In TRAM, a transformation is used, which maps a term with `n` sub-terms to a tree with `n+1` binary nodes. Since all nodes are now binary, no compactification is needed.

8.4.2 Persistence

Term rewriting is a functional paradigm: there is no assignment. This means when a binary node is created, both sub-trees already exist, and that node will never be changed.

We will use this property in the mark phase, in the recursive mark step, by traversing all nodes from youngest to oldest. **If a recent node isn't marked, it can not become marked by considering older nodes!**

To visit all nodes in order of creation (young to old) requires bookkeeping. In TRAM, every node has a hidden pointer to the next less recent node.

8.4.3 TRAM Garbage Collector

The TRAM garbage collector can be described as follows

```
mark stacks and registers
for all nodes from recent to old
    if the node is marked,
    mark its two children
    and keep the node in the 'in use' list
    or else
    move the node to the 'unused' list
```

The TRAM garbage collector uses a single pass in which Recursive Mark and Sweep are combined.

The space overhead is limited to a factor less than three:

- One pointer per binary node for ordering nodes from young to old
- Less than one pointer per term per sub-term to represent nodes in a binary tree

The time overhead is very small.

The source code of the garbage collector is less than 50 lines of (C) code.

Figure 8.3: Fragmentation

9. Binary Numbers

Tram does not offer built-in integers, which may seem an omission, but being unopinionated is at odds with built-in data types. However, numbers are part and parcel of computing, so they can't be missed. Elsewhere we have seen how sets can represent numbers (Peano numbers) and how terms can also be used to represent numbers. For instance, addition on `s-z` numbers is defined in the TRS

```
a(s(X),Y) = s(a(X,Y));
a(z,X) = X;
```

This TRS isn't suitable in practice. Imagine a system that needs to handle 32-bit values. A single term representing a large value has a depth in excess of that number and requires (in TRAM) many gigabytes of memory to represent. Furthermore, adding two such values requires a number of rewrite steps in the order of the depth.

The common solution to this is a positional numbering system as used in computers and in our daily lives. A positional numbering system uses a fixed set of digits (humans use ten, 0-9, and computers use two, 0-1). A number is then represented by a string of digits, where the position of each digit, usually from right to left, signifies its magnitude (each order of magnitude coincides with the number of digits). For instance, in `123`, the magnitude of `2` is ten times that of `3` and a tenth of that of `1`.

The same principle can be used in Tram, for instance to define binary numbers. We can use two constants `#0` and `#1` to represent binary digits, and we can use the function `bin` to represent strings of bits. Now, a number such as 163 (decimal, which is binary 10100011) can be represented as `bin(#1, bin(#0, bin(#1, bin(#0, bin(#0, bin(#0, bin(#1, #1)))))))`.

This may still seem like a significant overhead, but the size of a term representing a number is the same order (Big-O) as the number of bits. There exists no representation that improves upon this!

Before proceeding, we need to sanitize this specification because currently distinct values can be represented, which we regard as equal. For instance, `bin(#0,#0)`, the number 00, is really the same as `#0`. Secondly, we set out to define strings of bits, but so far, we defined trees of bits, which isn't the same. For instance, the number `101` could be represented with two trees:

`bin(#1,bin(#0,#1))` or `bin(bin(#1,#0),#1)`. One might say '*they are different notations of the same value*,' but we aimed to define a positional numbering system. In such a system the first notation, `bin(#1,bin(#0,#1))`, equals `3` (since `bin(#0,#1)` is `#1`) but `bin(bin(#1,#0),#1)` is `5`.

The rules to 'fix' (i.e. reduce) such spurious terms are:

```
bin(#0,X) = X;
bin(X,bin(Y,Z)) = bin(add(X,Y),Z);
```

Note that the second rule is intuitively obvious if one takes `bin(A, B)` in a binary positional numbering system to mean `2*A+B`. Then `bin(X,bin(Y,Z))` is `2*X+(2*Y+Z)`, which is `2*(X+Y)+Z`

How can one be certain these are the only spurious terms that might occur? There are two answers:

- The TRS described here also appears in (Walters & Zantema, 1995). That article proves that the TRS is confluent and terminating (which implies every term has a unique normal form) and that the normal forms coincide with the non-negative integers.
- A more intuitive answer is based on the following observation: every `bin`-term corresponds to a parenthesized binary number, i.e., a binary number in which as many balanced, meaningful parentheses have been inserted as possible (here, we call the outer parentheses in `1((01))` meaningless because they do not alter the tree-shape). For three bits, this results in the two strings `(10)1` and `1(01)`.
  If every right-associated string is reduced, only left-associated strings remain, and they coincide trivially with binary numbers.

This TRS requires the definition of addition:

```
add(#0,X) = X;
add(#1,#1) = bin(#1,#0);
add(bin(X,Y),Z) = bin(X,add(Y,Z));
```

Completeness of these rules (proven in (Walters & Zantema, 1995)) is intuitively clear when one considers

- The normal forms of this system are `#0`, `#1` and `bin(P, Q)`, for normal forms `P` and `Q`.
- `add` applied to a normal form can always be reduced.
- When it is reduced, the first argument of `add` is smaller (`Y` is smaller than `bin(X, Y)`)
- For any term, an innermost `add` can be reduced until it disappears
- By induction, any term containing `add` can be reduced until it is a normal form not containing `add`.

This blog is about implementation and not proof, but it makes sense to sketch how a programmer might avoid bugs leading to divergence or non-termination. The pattern is:

- *Determine the set of normal forms (i.e., intended irreducible terms)*
- *For every function which isn't a constructor of normal forms, ensure a rule exists for that function applied to all possible normal forms and that the term it is reduced to only contains the function applied to smaller terms*
- *More broadly, ensure a rule exists that reduces any non-normal form to some smaller form*

This formulation is a useful rule of thumb, but it is somewhat vague (what does 'smaller' mean), and it is incomplete in any event (bug-free systems exist which do not meet this property). If this were not the case, the study of confluence and termination mentioned in here would be straightforward.

In five rules, we have now defined binary numbers with addition. The number of bits required to store a value are the same order (differing only by a constant factor) as the term representation. The complexity of addition using this TRS is the same as that of adding binary numbers, although, admittedly, adding an `N`-bit number using this TRS takes `O(N)` steps whereas adding integers in a CPU takes a single clock cycle. However, that feat is only possible due to the massive level of parallelism in a CPU. Adding 64-bit numbers takes 64 small components to work in unison. It is a truism that everything in a computer is O[1].

To finalize this module, we add multiplication, which is entirely trivial.

```
mul(#0,N) = #0;
mul(#1,N) = N;
mul(bin(A,B),C) = bin(mul(A,C),mul(B,C));
```

The module `BinaryNumbers` in TRAM.1's test suite has a few additional rules:

- Generic function `eqn` is defined elsewhere
- Equality of data `#0` and `#1` is defined elsewhere
- Symmetrical rules for `add` are added for speed
- The more complex form of `mul` is used for speed
- `succ` is a short-hand

```
bin(#0,X) = X;
bin(X,bin(Y,Z)) = bin(add(X,Y),Z);

eq(bin(N,M),bin(P,Q)) = eqn(eq(N,P),M,Q);

add(#1,#1) = bin(#1,#0);
add(#0,X) = X;
add(X,#0) = X;
add(X,bin(Y,Z)) = bin(Y,add(X,Z));
add(bin(X,Y),Z) = bin(X,add(Y,Z));

mul(#0,N) = #0;       mul(N,#0) = #0;
mul(#1,N) = N;       mul(N,#1) = N;
mul(bin(A,B),bin(C,D))
   = bin(bin(mul(A,C),add(mul(A,D),mul(B,C))),mul(B,D));

succ(N) = add(N,#1);
```

10. TRAM.1

TRAM.1 is an implementation of term rewriting systems. TRAM.1 is written in standard C using only a few standard libraries. TRAM.1 is small (~650 lines of code). The memory manager and garbage collector are in a certain sense efficient. TRAM.1 contains a naive, handcrafted sunny-day parser (i.e., almost without error handling). The rewriting engine is naive: rules are attempted sequentially without much compilation or optimization.

TRAM.1 is available at Github

This section contains a rationale and explanation of TRAM.1, followed by a detailed user manual.

10.1 Rationale

TRAM.1 consists of the following parts (lines of code per part):

- The definition of `struct node` and some auxiliary macros (~30 loc)
- Declaration of global variables (~10 loc)
- Initialisation, in which memory is formatted (~10 loc)
- Garbage collector (~50 loc)
- Memory manager(~15 loc)
- The parser (~140 loc)
- Term printer ((~90 loc))
- Rewrite engine (~150 loc)
- CLI argument processor (~100 loc)

10.1.1 Tagged Values

TRAM.1 uses 32-bit ints to represent values (symbols, variables, references). The set of symbols is divided into a set 'Data', for which the engine assumes no rewrite rules exist, and other symbols. This means there are over 1 billion proper function symbols and as many constants to represent characters, (30 bit) integers, tiny floats, etcetera. Note that separating the set of symbols doesn't introduce magic. There are no built-in functions for integers, floats, etcetera.

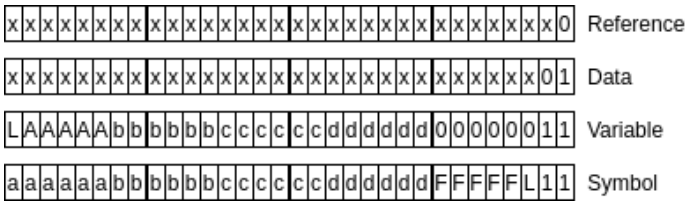


Figure 10.1: Tagged Values

TRAM.1 is intended to be self-contained, i.e., able to process term rewriting systems in source format (without a compiler) and to reduce terms either in term-notation or as textual or binary data.

The conflict of the limitation to 32-bit ints and the need to read and write identifiers is solved by encoding short identifiers. TRAM.1 encodes symbol names five characters maximum and variable names of four characters maximum in a 32-bit int. Data-symbols can be input as characters, integers, or hexadecimal numbers, but are represented as binary numbers (this means, `'A'`, `#65` and `#0x41` represent the same value and are output as `#0x41`, or `A` when using the appropriate CLI flag).

- `[@$a-z][A-Za-z0-9.]+ => Symbol`
  Only the first five characters are significant, so `abcdef` and `abcdefg` are considered identical. Identifiers shorter than 6 characters and identifiers longer than 5 characters are distinguished, so `abcde` and `abcdef` differ, but `abcdef` and `abcdeg` are identical and are output as `abcde...`
- `[*&A-Z][A-Za-z0-9.]+ => Variable`
  Only the first four characters are significant, using a similar mechanism as symbols
- `'#''0''x'[0-9a-fA-F]+ | '#''-'?[0-9]+ | ['].['] => Symbol`
  Data can be any decimal integer, hexadecimal number, or character.

10.1.2 Memory

Memory is allocated as a single block for a fixed number of nodes. This number has a default but can be set with a CLI argument.

On initialization, all nodes are pushed on the list of free nodes (linked through the `nxt` field). Creating a `new` node pops one off the free-list and pushes it on the `nxt`-linked list of used nodes, ready for garbage collection.

10.1.3 Garbage Collector

When a new node is created, but the list of free nodes is empty, the garbage collector is activated.

- First, all global variables are marked (marking uses an unused bit in the `nxt` field)
- Then, all used nodes are traversed.
  - If a node is marked, its children are marked, the node itself is unmarked and remains on the list of used nodes
  - if a node is unmarked, it is unused and is moved to the list of free nodes If the list of free nodes is still empty, memory has overflowed, and TRAM.1 exits. Otherwise, a new node can be created.

There are only a few global variables (called the **registers** of TRAM.1) that the garbage collector needs to be aware of:

- `P`, the current rewrite system 'program' (pointer to node)
- `S`, the stack (pointer to node)
- `X`, an auxiliary register (pointer to node)
- `T`, the register which holds a subject term
- `V`, the register which holds the current reduced value

In principle, local variables can be used to hold values or pointers to nodes, but: if a node `N` is pointed to by a local variable, and a new node is created, a garbage collection might be triggered,

and `N` wouldn't be recognized as a used node. For this reason, care is taken to store nodes only in the registers when `new` might be called (and to reset the registers to avoid floating garbage).

10.1.4 Scanner / Parser

The scanner/parser is a straightforward scanner for values. The following tokens deserve description:

- `(`: a symbol has been seen (the ofs of a term), and a term without arguments is pushed on the stack
- `,`: if a symbol has just been seen, it is apparently a term without sub-terms, and that term is added to the top-of-stack. Otherwise, the compound term just seen is added
- `)`: if a symbol has just been seen, it is apparently a term without sub-terms, and that term is added to the top-of-stack. Otherwise, the compound term just seen is added. Then, the top-of-stack is popped
- `=`: this means the *lhs* of a rule has just been seen; it is left on the stack.
- `;`: this means the *rhs* of a rule has just been seen. The pair of this rhs and the corresponding lhs is left on the stack. Note that this means the stack now holds nodes, which do not represent meta-terms. Representing a TRS as a meta-term at run-time wastes time and space
- `EOF`: either a term has been read (which is then returned), or the stack contains a linked list of pairs lhs/rhs in reversed textual order. Then this is returned.

10.1.5 Printers

TRAM.1 has three printers

- to print a value
- to print a term (uses X as a stack)
- to print a program (TRS). As mentioned, programs are stored in a more compact form than meta-terms, so they need their own printer

10.1.6 Rewrite Engine

The rewriting engine is closely related to the meta-interpreter in Section Term Rewriting, but as mentioned, there, the recursion and pattern matching must be implemented differently in C.

The engine can be described as a push-down automaton. It uses states to keep track of where it is in the algorithm. Each state 'knows' which variables are stored on the stack when it is entered.

Some states are:

- `BUDONECDR`: bottom-up rewriting, after the cdr has been normalized
- `TOPRED`: top-reduce, after all, sub-terms have been normalized
- `FORRULES`: iterate over the list of all rules

10.1.7 CLI Argument Processor

The argument processor is straightforward.

- Program (TRSs) and terms each can be read in one go (one file) or in chunks.
- flag `-P` (upper-case) reads the TRS at once, while repeated `-p` (lower-case) reads program fragments (= set of rules), after which `-C` joins the fragments. This allows programs to be maintained as a set of modules.
- flag `-T` (upper-case) reads a term at once, while repeated `-t` (lower-case) reads sub-terms, after which `-M` reads a meta-term. This is a term that may contain meta-variables *%n*, where `n` is the index of the n-th read sub-term. This way (for instance), a function can be applied to an input term
- in addition to terms, TRAM.1 can also read text files. Flag `-s` reads a text file and produces its term-representation (`str(...eos)`).

- flag `-I` prints the program (TRS) read
- flag -i print the subject term read
- flag `-r` reduces the term read using the program read
- flag `-O` prints the result after reduction. Note that debug settings influence how that term is represented (i.e., flattened or not)

10.2 User Manual

10.2.1 Tram.1 Syntax

Characters `*` and `&` in variables, and `$` and `@` in symbols have no special significance other than to express the special status of variables or symbols from the perspective of the programmer.

10.2.2 Tram.1 Semantics

Tram's semantics is described in Section Term Rewriting. Tram follows the right-most innermost reduction strategy. That is,

- given a term, first its right-most innermost sub-term is considered for reduction
- then the left-hand sides of all rules are matched against that sub-term
- rules are attempted in the order in which they have been read, either using the `-P` flag or in the order of modules (`-p` flag) followed by the order within that module
- if a match is found and the substitution is established, the corresponding right-hand side is instantiated using that substitution
  - TRAM.1 assumes no variables occur multiple times on the left-hand side. If this occurs, the value matched against the last (right-most) occurrence is used
  - TRAM assumes that for every variable on the right-hand side, a value has been matched on the left-hand side. This isn't checked; using an unmatched variable leads to erroneous results
  - The result of this instantiation is further normalized
  - Then, the next right-most innermost term is attempted
- if no match is found, the sub-term is built, and the next right-most, innermost sub-term is considered
- this process continues until the entire subject term has been processed. The result is the now normalized subject term ## TRAM.1 CLI TRAM.1 accepts the following CLI arguments

```
    Note that D and X must appear as 1st/2nd cli argument when used
-D n          generate level n (1..3) debugging info
              Level 0: none
     Level 1: only I/O (flat string repr.)
     Level 2: Garbage Collection
     Level 3: Reduction step information
     Level 4: Engine cycle information
-X nnn        set memory size to nnn

-P <fname>    Load program
-p <fname>    Load program segment
-C            Compile all segments into one program
      Note that either -P or -p...-p -C should be used
-T <fname>    Read term as subject
-t <fname>    Read sub-term
-s <fname>    Read string sub-term
```

```
-b <fname> <todo> Read binary sub-term
-M <fname>   Read meta-term as subject
-m <fname> <todo> Read meta term as sub-term
-I <fname>   dump program
-i <fname>   dump subject term
-r           reduce subject using program
-O <fname>   dump result term
-S <fname> <todo> print result as text
-B <fname> <todo> print result as binary
```

10.2.3 Formats

TRAM.1 accepts these input formats:

- Source
  The TRAM.1 source language for terms and term rewriting systems;
- Text
  Any text stream (arbitrary text terminated by EOF). The text "ccccc...", where 'c' are arbitrary characters, is rendered as the term `str('c', str('c', ...str('c',eos)...))`;
- Meta-Source
  This language is the same as the source language with one extension: meta-variables `'%'` `[0-9]+`. As input is read, each source, text or binary stream is rendered as a sub-term and is kept in a cache. As a meta-source input is read and the corresponding term is built, each meta-variable *%n* is immediately replaced by the `n`-th previously read term. Example: to transform a text file using some function, TRAM.1 might be called with arguments `-P <transformationProgram> -s <textToTransform> -M <meta>`, where the meta-text is `transform(%1)`.

10.2.4 Examples

The TRAM.1 distribution contains a large number of examples.

10.2.5 Distribution

The TRAM.1 distribution at Github contains the following files:

- `TRAM.1.c`
  The single C source file. There are no headers; TRAM.1 only includes `stdio.h`, `stdlib.h` and `stdint.h`.
- `test`
  A **bash** shell script containing many tests. The script
  - creates test files
  - executes TRAM
  - compares the output with the expected output
- `Makefile`
  The Makefile handles three commands:
  - `make TRAM` compiles and is dependent on TRAM.1.c
  - `make test` executes the test , which creates many test files and executes TRAM, and compares the output with pre-defined output
  - `make clean` removes all files generated by the test script

10.2.6 Installation

Requirements: `gcc`, `sh`

Execute: `make test`

11. Annotated TRAM.1

This section annotates the full source of TRAM.1 (version of Oct 2021).

11.1 Basics

11.1.1 Standard libraries

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
```

11.1.2 Single Error Logger

```
#define error(...) {\
    fprintf( stderr, "***ERROR***\n"); \
    fprintf( stderr, __VA_ARGS__); \
    fprintf( stderr, "\n"); \
    fflush(stderr); \
    exit(1); \
}
```

11.1.3 Tagged Value Type

```
typedef uint32_t tval; // tagged value

// Layered tagged values:
// x0 ref (idx)
// d01 dta (d is int, char, bin/dec/hex digit, small float,
    ...)
```

```
// LVvvv0..011 var: V=[*&A-Z], v=[^A-Za-z0-9] or null (right-to-left), L=1 if longer than 4
// ffffFL11 id, F=[@\mydollarA-Z], f=[^A-Za-z0-9] or null (right-to-left), L=1 if longer than 5
```

11.1.4 Predicates for Node, Variable, Symbol

```
#define isREF(t) (((t)&1)==0)
#define isNREF(t) (((t)&1)==1)
#define isVAR(t) (((t)&0xff)==3)
#define isFUN(t) (((t)&3==3)&&((t)&0xff)>3)
```

11.1.5 Conversion Pointer <==> Reference-value

```
#define ref(t) (mem+(t)/2)
#define idx(r) ((r-mem)*2)
```

11.1.6 Node Structure

```
typedef struct _node {
    tval car, cdr, nxt;
} node;
typedef node *ref;
```

11.1.7 Forward Declaration

```
ref new(tval x,tval y);
```

11.1.8 Stack Manipulation

```
#define Push(X,a) X=new(a,idx(X));
#define Pop(X) X->car; X=ref(X->cdr);
#define PopRef(X) ref(X->car); X=ref(X->cdr);
```

11.1.9 Declaration of Memory and Memory Management Variables

```
int MEMSIZE=100000; //nodes
ref mem, usedNodes, freeNodes;
#define nil mem
```

11.1.10 Debugging Levels

```
int Dbg=0;
enum DbgSt {DNone,DIO,DGC,DSteps,DStepDump,DCycles};
//debug levels -- 0: none, 1: I/O, 2: gc, 3: rewr steps, 4:: dump steps, 5:dump cycles
```

11.1.11 Counters for Logging

```
int Drulei,Drewrcnt=0;
```

11.2 Memory Management and Garbage Collector

11.2.1 GC Registers

```
//registers safe from GC
ref P, S, X;
tval T, V;
```

11.2.2 Memory Initialisation

```
void init() {
    if (sizeof(tval)!=4 || sizeof(node)!=12) {
        error("Bad size tval: %ld (=4), sizeof node: %ld
   (=12)\n",sizeof(tval), sizeof(node))
    }
    mem = (ref)malloc(MEMSIZE * sizeof(node));
    mem -= 1; // index 0 is nil and not a node
 freeNodes = nil;
    for (ref r=mem+1; r-mem<MEMSIZE; r++) {
        r->nxt=idx(freeNodes);
        freeNodes = r;
    }
    P = S = X = usedNodes = nil;
    T = V = 0;
}
```

11.2.3 Node Bit Manipulation

```
#define MARK 1
#define set(t,n) ((n)|=(t))
#define has(t,n) ((n)&(t))
#define rst(t,n) ((n)&=(~(t)))
```

11.2.4 Garbage Collection

Memory Management is discussed in Section Memory Management.

```
int gen=0;
void gc() {
   if (Dbg>=DGC) fprintf( stdout, "GC gen %d: ",gen++);
   ref trail=nil;
   long cnt = 0;

    if (P!=nil) set(MARK,P->nxt);
    if (X!=nil) set(MARK,X->nxt);
```

```
   if (S!=nil) set(MARK,S->nxt);
   if (isREF(T)&&T!=0) set(MARK,ref(T)->nxt);
   if (isREF(V)&&V!=0) set(MARK,ref(V)->nxt);

   for (ref r = usedNodes; r!=nil; ) {
      if (has(MARK,r->nxt)) {
         rst(MARK,r->nxt);
         if (isREF(r->car)) {
            set(MARK,ref(r->car)->nxt);
         }
         if (isREF(r->cdr)) {
            set(MARK,ref(r->cdr)->nxt);
         }
         if (trail==nil) {
            usedNodes = trail = r;
         } else {
            trail = r;
         }
         r=ref(r->nxt);
      } else {
         ref nxt;
         if (trail==nil) {
            usedNodes = nxt = ref(r->nxt);
         } else {
            trail->nxt = r->nxt;
                 nxt = ref(r->nxt);
            }
         r->nxt = idx(freeNodes);
         freeNodes = r;
         cnt++;
         r = nxt;
      }
   }
   if (Dbg>=DGC) fprintf( stdout, "%ld freed\n",cnt);
}
```

11.2.5 New Node

```
ref new(tval x, tval y) {
    if (freeNodes==nil) {
        gc();
    }
    if (freeNodes==nil) {
        error("Out of Memory");
    }
    ref tmp = freeNodes;
    freeNodes = ref(freeNodes->nxt);
    tmp->nxt = idx(usedNodes);
    usedNodes=tmp;
```

```
    tmp->car = x;
    tmp->cdr = y;
    return tmp;
}
```

11.3 I/O

11.3.1 Forward Declaration

```
void pval(tval v,int nl);
```

11.3.2 Scanner / Parser

Scanner / Parser discussed in Section Converting from C to Tram (Scanner/Parser)

```
FILE *in;
int c;
tval readTerm(char *nm) {
    tval num, v;
    int sgn, len;
    if ((in = fopen(nm, "r"))==NULL) {
        error("unknown file %s",nm);
    };
    c = fgetc(in);
    for (;;) {
        switch (c) {
            case ' ': case '\n': case '\r': case '\t': case '\v': case '\f':
                break;
            case '!': // comment
 while ((c=fgetc(in))!='\n') {}
                break;
             case '#': // data (hex or dec)
 if ((c=fgetc(in))=='0') {
                    if ((c=fgetc(in))=='x') {
                        num = 0;
                        while ((c = fgetc(in)) >= '0' && c <= '9' || c >= 'a' && c <= 'f' || c >= 'A' && c <= 'F') {
                            num = 16 * num + (c <= '9' ? c - '0' : c <= 'F' ? 10 + c - 'A' : 10 + c - 'a');
                        }
                        v = (num << 2) | 1;
                        continue;
                    } else if (c>='0'&&c<='9') {
                        sgn = 1;
                        num = c-'0';
                        goto decimal;
                    } else {
                            v = 1;
                            continue;
```

```
                    }
                } else {
                    if (c=='-') {
                        sgn = -1;
                        num = 0;
                    } else {
                        sgn = 1;
                        num = c-'0';
                    }
                  decimal:
                    while ((c=fgetc(in))>='0'&&c<='9') {
                        num = 10*num+c-'0';
                    }
                    v = ((sgn*num)<<2)|1;
                    continue;
                }
            case '\'': //data (char)
v = (fgetc(in)<<2)|1;
                if ((c=fgetc(in))!='\'') error("expected '
  got character %c (%d)",c,c);
                break;
            case '%': // dec (meta-variable)
num = 0;
                while ((c=fgetc(in))>='0'&&c<='9') {
                    num = 10*num+c-'0';
                }
                for (v=T, len=0; v!=0; v=ref(v)->cdr) len++;
                num = len-num;
                for (v=T; num--; v=ref(v)->cdr) {}
                v = ref(v)->car;
                continue;
            case '*': case '&': // Var
case 'A': case 'B': case 'C': case 'D': case 'E': case 'F':
  case 'G':
            case 'H': case 'I': case 'J': case 'K': case 'L':
  case 'M': case 'N':
            case 'O': case 'P': case 'Q': case 'R': case 'S':
  case 'T': case 'U':
            case 'V': case 'W': case 'X': case 'Y': case 'Z':
                len = 1;
                num = c=='*'?1:c=='&'?2:c-'A'+3;
                while
  ((c=fgetc(in))=='.'||c>='0'&&c<='9'||c>='a'&&c<='z'||c>='A'&&c<='Z')
  {
                    if (++len<=4) {
                        num = num << 6 |
            (c == '^' ? 1 : c - (c <= '9' ? '0'-2 : (c <= 'Z'
  ? 'A'-12 : 'a'-38)));
                    }
```

```
                }
                sgn = len>4?1<<31:0;
                while (++len<=4) {
                    num <<= 6;
                }
                v = (num<<8)|sgn|3;
                continue;
            case '@': case '\mydollar': // Id
case 'a': case 'b': case 'c': case 'd': case 'e': case 'f': case 'g':
            case 'h': case 'i': case 'j': case 'k': case 'l': case 'm': case 'n':
            case 'o': case 'p': case 'q': case 'r': case 's': case 't': case 'u':
            case 'v': case 'w': case 'x': case 'y': case 'z':
                len = 1;
                num = c=='@'?1:c=='\mydollar'?2:c-'a'+3;
                while ((c=fgetc(in))=='.'||c>='0'&&c<='9'||c>='a'&&c<='z'||c>='A'&&c<='Z') {
                    if (++len<=5) {
                        num = num << 6 |
            (c == '^' ? 1 : c - (c <= '9' ? '0'-2 : (c <= 'Z' ? 'A'-12 : 'a'-38)));
                    }
                }
                sgn = len>5?4:0;
                while (++len<=5) {
                    num <<= 6;
                }
                v = (num<<8) | (num>>21)&0xf8 | sgn | 3;
                continue;
            case '(':
                Push(S, idx(new(v,0)));
                v=0;
                break;
            case ',':
                if (isFUN(v)) v=idx(new(v,0));
                V = Pop(S);
                Push(S, idx(new(v,V)));
                V=v=0;
                break;
            case ')':
                if (isFUN(v)) v=idx(new(v,0));
                v=idx(new(v,0));
                V = Pop(S);
                while (V!=0) {
                    v=idx(new(ref(V)->car,v));
                    V = ref(V)->cdr;
```

```
                }
                break;
            case '=':
                if (isFUN(v)) v=idx(new(v,0));
                Push(S,v);
                v=0;
                break;
            case ';':
                if (isFUN(v)) v=idx(new(v,0));
                V = Pop(S);
                Push(S, idx(new(V,v)));
                v=0; V=0;
                break;
            case EOF:
                if (S!=nil) {
                    v=idx(S);
                    S=nil;
                }
                if (isFUN(v)) v=idx(new(v,0));
                return v;
            default:
                error("unexpected character %c (%d)",c,c);
        };
        c = fgetc(in);
    }
}
```

11.3.3 Rewrite Engine States

These values are used to represent states of the rewrite engine. For debugging purposes the print-names of those states are defined.

```
enum states {BURED=1000, BUDONECDR, BUDONEBOTH, TOPRED, FORRULES, MATCH, MATCHDONE, MATCHDONECAR,
    INSTDONECAR, INSTDONEBOTH, BUILD, ALLDONE, INST, INSTCONT, PRINT, PRT, PRINTDONECAR, PRINTDONEBOTH};
const char * prstates[] = {"BURED", "BUDONECDR", "BUDONEBOTH", "TOPRED", "FORRULES", "MATCH", "MATCHDONE", "MATCHDONECAR",
    "INSTDONECAR", "INSTDONEBOTH", "BUILD", "ALLDONE", "INST", "INSTCONT"};
```

11.3.4 Conversion for Data Values

```
#define asDTA(d)  1+4*d
#define PopDTA(X) (X->car-1)>>2; X=ref(X->cdr);
```

11.3.5 Function Symbol Constants

These constants are used by the CLI-interpreter in conversion from string to term representation. These aren't magic constants, but rather pre-encoded symbols `eos`, `str`, `lst` and `eol`.

```
#define FSYMeos 0xD380003B
#define FSYMstr 0xE77000AB
#define FSYMlst 0xE3900073
#define FSYMeol 0xD310003B
```

11.3.6 Identifier Decoding

Auxiliary values to decode identifiers

```
char *var0="~*&ABCDEFGHIJKLMNOPQRSTUVWXYZ",
*idc="~^0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz",
*fun0="~@\mydollarabcdefghijklmnopqrstuvwxyz";
```

11.3.7 Optionally Print

'Short' identifiers are filled with zeroes; `qpc` prints a character only if it's index isn't zero.

```
void qpc(char const c) {
    if (c!='~') fprintf(stdout, "%c",c);
}
```

11.3.8 Print Value (Reference, Date, Variable or Symbol)

```
void pval(tval v,int nl) {
    if ((v&1)==0) {//ref
 fprintf(stdout, "\\%d",v);
    } else if ((v&3)==1) {//data
 fprintf(stdout, "#0x%x",v>>2);
    } else if ((v&0xff)==3) {//Var
 fprintf(stdout, "%c",var0[(v>>26)&31]);
        qpc(idc[(v>>20)&63]); qpc(idc[(v>>14)&63]);
   qpc(idc[(v>>8)&63]);
        fprintf(stdout, "%s",(v>>31)&1?"...":"");
    } else {//Id
 fprintf(stdout, "%c",fun0[(v>>3)&31]);
        qpc(idc[(v>>26)&63]); qpc(idc[(v>>20)&63]);
   qpc(idc[(v>>14)&63]); qpc(idc[(v>>8)&63]);
        fprintf(stdout, "%s",((v>>2)&1)?"...":"");
    }
    if (nl) fprintf(stdout, "\n");
}
```

11.3.9 Print Term

Print a term; `flats` requests flat strings, i.e., terms such as `str('o',str('n',str('e',eos)))` to be printed as `"one"` to help in debugging.

```
int flats=0;
void ptrm(tval t) {
    ref ptr;
    X=nil;
    int instr = 0;
    for(;;) {
        if (isREF(t)) {
            ptr = ref(t);
            if (isFUN(ptr->car)) {
                if (flats && ptr->car == FSYMstr) {
                    if (!instr) {
                        instr = 1;
                        fprintf(stdout, "\"");
                    }
                    fprintf(stdout, "%c",
   (char)((ref(ref(ptr->cdr)->car)->car) >> 2));
                    t = ref(ref(ptr->cdr)->cdr)->car;
                    continue;
                } else if (flats && ptr->car == FSYMeos) {
                    if (instr) {
                        instr = 0;
                        fprintf(stdout, "\"");
                    } else {
                        fprintf(stdout, "\"\"");
                    }
                } else {
                    pval(ptr->car,0);
                    if (ptr->cdr != 0) {
                        fprintf(stdout, "(");
                        Push(X, (')' << 2) + 1);
                        t = ptr->cdr;
                        continue;
                    }
                }
            } else {
                if (ptr->cdr!=0) {
                    Push(X,ptr->cdr);
                    Push(X,(',’<<2)+1);
                }
                t = ptr->car;
                continue;
            }
        } else {
            pval(t,0);
        }
        while (X!=nil && isNREF(X->car)) {
            t=Pop(X);
            fprintf(stdout, "%c", t>>2);
        }
```

```
        if (X==nil) return;
        t = Pop(X);
        continue;
    }
}
```

11.3.10 Print TRS

```
void pprg(ref p) {
    while(p!=nil) {
        ptrm(ref(p->car)->car);
        fprintf(stdout, " = ");
        ptrm(ref(p->car)->cdr);
        fprintf(stdout, ";\n");
        p=ref(p->cdr);
    }
    fprintf(stdout, "\n");
}
```

11.4 Rewrite Engine

Discussed in Section Converting Rewrite Engine from Tram to C

```
ref reduce (/*tval T, ref P*/) {
    S = nil;
    int state=BURED;
    tval f, t, pat;
    ref p, sub;
    Push(S,asDTA(ALLDONE));
loop: //full-reduce is BURED
 if (Dbg>=DCycles) fprintf(stdout, "%s
   ",prstates[state-1000]);
    switch (state) { // T is subject term, V is result
 case BURED: //T is term
 if (isNREF(T)||T==0) {
                V = T;
                state = PopDTA(S);
                goto loop;
            }
            Push(S,ref(T)->car);
            Push(S,asDTA(BUDONECDR));
            T=ref(T)->cdr;
            goto loop;
        case BUDONECDR: //V is cdr, tos is car-to-do
 T = Pop(S);
            Push(S,V);
            Push(S,asDTA(BUDONEBOTH));
            state = BURED;
            goto loop;
```

```
        case BUDONEBOTH: //(V is car), tos is cdr
if (isREF(V)) {
                T = Pop(S);
                V = idx(new(V,T));
                state = PopDTA(S);
                goto loop;
            }
            f = V;
            T = Pop(S);
            state = TOPRED;
            goto loop;
        case TOPRED: //(f is fun,T = args)
if (Dbg>=DStepDump) pval(f,1);
            p = P;
            t = T;
            state = FORRULES;
            if (Dbg>=DSteps) Drulei=0;
            goto loop;
        case FORRULES: //(f,t,p, T, P)
if (p==nil) { // eor
state = BUILD;
                goto loop;
            }
            if (Dbg>=DSteps) Drulei++;
            if (ref(ref(p->car)->car)->car != f) {
                p=ref(p->cdr);
                goto loop;
            }
            sub = nil;
            Push(S,asDTA(MATCHDONE));
            state = MATCH;
            pat = ref(ref(p->car)->car)->cdr;
            goto loop;
        case MATCH: //(t,pat,sub,p, T, P)
if (isREF(pat)&&pat!=0) { // compound
if (isNREF(t)) goto fail;
                Push(S,ref(t)->cdr);
                Push(S,ref(pat)->cdr);
                Push(S,asDTA(MATCHDONECAR));
                pat=ref(pat)->car;
                t=ref(t)->car;
                goto loop;
            }
            if (isVAR(pat)) {
                sub = new(idx(new(pat,t)),idx(sub));
                state = PopDTA(S);
                goto loop;
            }
            if (pat==t) {
```

```
                state = PopDTA(S);
                goto loop;
            }
            //fallthrough intended
fail:
            p=ref(p->cdr);
            t = T;
            do {state = PopDTA(S);
            } while(state!=MATCHDONE);
            state = FORRULES;
            goto loop;
        case MATCHDONECAR://(sub,p) tos is pat.cdr, 2nd is
  trm.cdr
pat = Pop(S);
            t = Pop(S);
            state = MATCH;
            goto loop;
        case MATCHDONE://(sub,p)
pat = ref(p->car)->cdr; //rhs
state = INST;
            if (Dbg>=DSteps) Drewrcnt++;
            if (Dbg>=DSteps) printf("Rule %d, Steps
  %d\n",Drulei,Drewrcnt);
            goto loop;
        case INST: //(pat,sub)
if (isREF(pat)&&pat!=0) {
                Push(S,ref(pat)->cdr);
                pat=ref(pat)->car;
                Push(S,asDTA(INSTDONECAR));
                goto loop;
            }
            if (isVAR(pat)) {// use p as tmp in sub
p = sub;
                while (ref(p->car)->car!=pat) {
                    p = ref(p->cdr);
                }
                V = ref(p->car)->cdr;
                state = PopDTA(S);
                goto loop;
            }
            V = pat;
            state = PopDTA(S);
            goto loop;
        case INSTDONECAR: // V is car
pat = Pop(S);
            Push(S,V);
            Push(S,asDTA(INSTDONEBOTH));
            state = INST;
            goto loop;
```

```
        case INSTDONEBOTH: // V is cdr, ToS = car
 t = Pop(S);
            if (isFUN(t)) {
                f = t;
                T = V;
                Push(S,pat);
                Push(S,idx(sub));
                Push(S,asDTA(INSTCONT));
                state = TOPRED;
                goto loop;
            }
            V = idx(new(t,V));
            state = PopDTA(S);
            goto loop;
        case INSTCONT: {// V is car
 sub = PopRef(S);
            pat = Pop(S);
            state = PopDTA(S);
            goto loop;
            }
        case BUILD:
            V = idx(new(f,T));
            state = PopDTA(S);
            goto loop;
        case ALLDONE: //done
 //res = Pop(S); if (S!=nil) {
                error("Stack should be empty!");
            }
            return ref(V);
        default: error("Bad state!");
    }
}
```

11.4.1 rcat, rev

Auxiliary functions to reverse a string or list.

```
ref rcat(ref r, ref s) {//prepend reverse r to s
 while (r!=nil) {
        Push(s,r->car);
        r=ref(r->cdr);
    }
    return s;
}
#define rev(r) rcat(r,nil)
```

11.5 CLI Interpreter

```
int main(int argc, char *argv[]) {
```

```
    int i=1;
    tval t;
    char *nm;
    int fln,fi=0;

    if (argc<2) {
        printf(
        "      Note that D and X must appear as 1st and 2nd cli switch when used\n"
"-D n          generate level n (1..3) debugging info\n"
"-X nnn        set memory size to nnn*1000\n"
"\n"
"-P <fname>    Load program\n"
"-p <fname>    Load program segment\n"
"-C            Compile all segments into one program\n"
"      Note that either -P or -p...-p -C should be used\n"
"-T <fname>    Read term as subject\n"
"-t <fname>    Read sub-term\n"
"-s <fname>    Read string sub-term\n"
"-b <fname> <todo> Read binary sub-term\n"
"-M <fname>    Read meta-term as subject\n"
"-m <fname> <todo> Read meta term as sub-term\n"
"-I <fname>    dump program\n"
"-i <fname>    dump subject term\n"
"-r            reduce subject using program\n"
"-O <fname>    dump result term\n"
"-S <fname> <todo> print result as text\n"
"-B <fname> <todo> print result as binary\n"
);
    }
    if (i<argc && argv[i][1]=='D') { Dbg = argv[++i][0]-'0'; i++; }
    if (i<argc && argv[i][1]=='X') { sscanf(argv[++i], "%i", &MEMSIZE); MEMSIZE*=100000; i++; }

    init();

    P=nil;
    V=0;
    while (i<argc && argv[i][0]=='-') {
        switch (argv[i++][1]) {
            case 'P':   P = rev(ref(readTerm(argv[i++])));
                        break;
            case 'p':   P = new(readTerm(argv[i++]),idx(P));
                        break;
            case 'C':   for (X=nil; P!=nil; ) {
                            t = Pop(P);
                            while ( t!=0) {
                                Push(X,ref(t)->car);
```

```
                                t = ref(t)->cdr;
                            }
                        }
                        P = X; X=nil;
                        break;
            case 'T':
            case 'M':   T = readTerm(argv[i++]);
                        break;
            case 't':   T = idx(new(readTerm(argv[i++]), T));
                        break;
            case 's':   if ((in = fopen(argv[i++], "r"))==NULL) {
                            error("unknown file %s",argv[i++]);
                        };
                        fseek(in,0,SEEK_END);
                        fln=ftell(in);
                        V = idx(new(FSYMeos,0));
                        for(fi=1;fi<=fln;fi++) {
                            fseek(in,-fi,SEEK_END);
                            c = fgetc(in);
                            V = idx(new(V,0));
                            V = idx(new(idx(new((c<<2)|1,0)),V));
                            V = idx(new(FSYMstr,V));
                        }
                        fclose(in);
                        T = idx(new(V, T));
                        break;
            case 'b':
            case 'm':   error("not yet implemented");
            case 'I':   pprg(P);
                        printf("\n");
                        break;
            case 'i':   if (Dbg>=DIO) flats=1;
                        ptrm(T);
                        printf("\n");
                        flats=0;
                        break;
            case 'r':   if (Dbg>=DIO) printf("\n### Start ###\n");
                        X = reduce(); //uses T
if (Dbg>=DIO) printf("### Done! ###\n");
                        break;
            case 'O':   if (Dbg>=DIO) flats=1;
                        ptrm(idx(X));
                        printf("\n");
                        flats=0;
                        break;
```

```
            default:    error("No such option: %c. Use %s for help.",argv[i-1][1],argv[0]);
        }
    }
}
```

III How to Write TRS's

12 Converting C to Tram 73
12.1 Scanner / Parser in Tram

13 Converting Tram To C 83
13.1 Preamble
13.2 TRAM.1's Rewrite Engine

14 Patterns: Encoding-Checker from Scratch 93
14.1 Pattern: Initializing Local Variables
14.2 Pattern: Function Name or Extra Parameter
14.3 Pattern: Have Your Cake

12. Converting C to Tram

This section describes how TRAM's scanner / parser written in C was used as a basis to implement the scanner / parser in Tram.

12.0.1 Annotated Scanner / Parser in C

The scanner is a loop which processes input characters and terminates when EOF is read. Inside the loop a `switch` branches based on the current character under consideration.

Variables:

- `v` holds the currently value being read
- `num` and `sgn` are auxiliary variable used to read numbers
- `len` is an auxiliary variable used to read identifiers
- `in` and `nm` are file name and descriptor
- `S` is used as a stack and has been initialized in the context
- register `V` is used as an auxiliary register to safeguard terms from garbage collection
- register `T` contains a list of sub-terms read so far using `-s`

Note: a term can be a variable, a compound term (`tram(...)`), or a sole function symbol `f` which represents a term with zero arguments. In various places where the parser expects a term, but where the scanner has seen a symbol, that symbol must be coerced to a term (by replacing `f` with `new(f,0)`). Wherever the text mentions a (possibly coerced) term, this mechanism is intended without further explanation (the code is self-evident).

```
tval readTerm(char *nm) {
    tval num, v;
    int sgn, len;
    if ((in = fopen(nm, "r"))==NULL) {
        error("unknown file %s",nm);
    };
    c = fgetc(in);
    for (;;) {
 switch (c) {
```

Note: immediately after the switch statement is the single statement:

```
    }
    c = fgetc(in);
    }
```

This means that the `break` statement results in the next character being read and the loop restarted, whereas `continue` restarts the loop without reading a character (i.e., when the next character has already been read).

Whitespace and comments are simply skipped.

```
    case ' ': case '\n': case '\r': case '\t':
    case '\v': case '\f':
    break;
    case '!': // comment
    while ((c=fgetc(in))!='\n') {}
    break;
```

A quoted character is immediately encoded to a corresponding data value.

```
    case '\'': //data (char)
    v = (fgetc(in)<<2)|1;
    if ((c=fgetc(in))!='\'')
    error("expected ' got character %c (%d)",c,c);
    break;
```

A # can be followed by

- 0x followed by a hexadecimal value, or
- a decimal value. A 0 not followed by x is the beginning of a decimal value (`goto` is used to jump into the decimal handler). The encoding data value is computed on the fly.

```
case '#': // data (hex or dec)
    if ((c=fgetc(in))=='0') {
       if ((c=fgetc(in))=='x') {
       num = 0;
       while ((c = fgetc(in)) >= '0' && c <= '9'
               || c >= 'a' && c <= 'f' || c >= 'A' && c <=
    'F') {
       num = 16 * num +
             (c <= '9' ? c - '0'
                : c <= 'F' ? 10 + c - 'A' : 10 + c - 'a');
       }
       v = (num << 2) | 1;
       continue;
       } else if (c>='0'&&c<='9') {
       sgn = 1;
       num = c-'0';
       goto decimal;
       } else {
       v = 1;
       continue;
       }
    } else {
```

```
      if (c=='-') {
      sgn = -1;
      num = 0;
      } else {
      sgn = 1;
      num = c-'0';
      }
    decimal:
      while ((c=fgetc(in))>='0'&&c<='9') {
      num = 10*num+c-'0';
      }
      v = ((sgn*num)<<2)|1;
      continue;
    }
```

Meta variables contain a decimal number identifying the n-th element (in reverse order) from the list in `T`. This value is returned as the value just read.

```
case '%': // dec (meta-variable)
    num = 0;
    while ((c=fgetc(in))>='0'&&c<='9') {
    num = 10*num+c-'0';
    }
    for (v=T, len=0; v!=0; v=ref(v)->cdr) len++;
    num = len-num;
    for (v=T; num--; v=ref(v)->cdr) {}
    v = ref(v)->car;
    continue;
```

A Tram variable consists of `*`, `&` or an upper-case letter, followed by between zero and three letters, digits or `.`. Only the first four characters are significant, although variables shorter than 5 characters and variables longer than 4 characters are distinguished, so `Abcd` and `Abcde` differ, but `Abcde` and `Abcdf` are considered identical.

A symbol consists of `$`, `@` or a lower-case letter, followed by between zero and four letters, digits or `.`. Only the first five characters are significant, although symbols shorter than 6 characters and symbols longer than 5 characters are distinguished, so `abcde` and `abcdef` differ, but `abcdef` and `abcdeg` are considered identical.

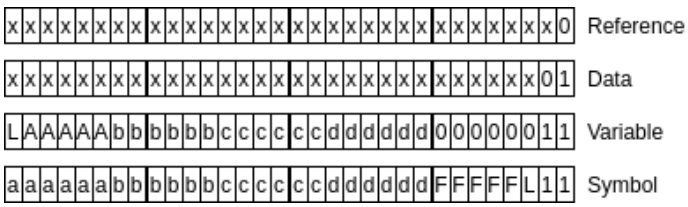


Figure 12.1: Tagged Values

Only the first five characters are significant, so `abcde` and `abcdef` differ, but `abcdef` and `abcdeg` are considered identical.

- In a variable or symbol the L bit signifies whether the identifier is longer than the max (4 for variables, 5 for symbols)
- `AAAAA` in a variable, or `FFFFF` in a symbol encode the first character
- `aaaaaa`, `bbbbbb`, etcetera encode subsequent characters.

```
case '*': case '&': // Var
```

```
2 case 'A': case 'B': case 'C': case 'D': case 'E': case 'F':
    case 'G':
3 case 'H': case 'I': case 'J': case 'K': case 'L': case 'M':
    case 'N':
4 case 'O': case 'P': case 'Q': case 'R': case 'S': case 'T':
    case 'U':
5 case 'V': case 'W': case 'X': case 'Y': case 'Z':
6     len = 1;
7     num = c=='*'?1:c=='&'?2:c-'A'+3;
8     while ((c=fgetc(in))=='.'||c>='0'&&c<='9'
9            ||c>='a'&&c<='z'||c>='A'&&c<='Z') {
10        if (++len<=4) {
11            num = num << 6 | (c == '^' ? 1
12      : c - (c <= '9' ? '0'-2 : (c <= 'Z' ? 'A'-12 :
    'a'-38)));
13        }
14    }
15    sgn = len>4?1<<31:0;
16    while (++len<=4) {
17        num <<= 6;
18    }
19    v = (num<<8)|sgn|3;
20    continue;
21 case '@': case '\mydollar': // Id
22 case 'a': case 'b': case 'c': case 'd': case 'e': case 'f':
    case 'g':
23 case 'h': case 'i': case 'j': case 'k': case 'l': case 'm':
    case 'n':
24 case 'o': case 'p': case 'q': case 'r': case 's': case 't':
    case 'u':
25 case 'v': case 'w': case 'x': case 'y': case 'z':
26    len = 1;
27    num = c=='@'?1:c=='\mydollar'?2:c-'a'+3;
28    while ((c=fgetc(in))=='.'||c>='0'&&c<='9'
29           ||c>='a'&&c<='z'||c>='A'&&c<='Z') {
30        if (++len<=5) {
31            num = num << 6 | (c == '^' ? 1
32    : c - (c <= '9' ? '0'-2 : (c <= 'Z' ? 'A'-12 : 'a'-38)));
33        }
34    }
35    sgn = len>5?4:0;
36    while (++len<=5) {
37        num <<= 6;
38    }
39    v = (num<<8) | (num>>21)&0xf8 | sgn | 3;
40    continue;
```

So far, we have seen a scanner and not a parser. The only feature in Tram that isn't lexical and is context free are parentheses. The scanner / parser is a push-down automaton which uses a stack (S) to hold partial results.

When a `(` is encountered, a symbol must just have been seen. The current value is kept in variable `v`, so a term with `v` as outermost function symbol is pushed on the stack (`new(v,0)`). This term will be extended as arguments are parsed.

```
case '(':
    Push(S, idx(new(v,0)));
    v=0;
    break;
```

A `,` means a (possibly coerced) term has just been read. That term is pushed on whatever partial term is on the stack. These pushed sub-terms therefore occur in the wrong order. We will fix this at the closing `)`. Note that `V` is cleared to avoid floating garbage, and that `v` is cleared because there is no current read object.

```
case ',':
    if (isFUN(v)) v=idx(new(v,0));
    V = Pop(S);
    Push(S, idx(new(v,V)));
    V=v=0;
    break;
```

Just like a `,`, a `)` immediately follows a (possibly coerced) term. However, that structure is in the wrong order: The last argument is on top, and the function symbols are deepest. A new object is created in which all parts occur in the right order.

```
case ')':
    if (isFUN(v)) v=idx(new(v,0));
    v=idx(new(v,0));
    V = Pop(S);
    while (V!=0) {
    v=idx(new(ref(V)->car,v));
    V = ref(V)->cdr;
    }
    break;
```

`=` occurs in a rule, but otherwise behaves as a `,`.

```
case '=':
    if (isFUN(v)) v=idx(new(v,0));
    Push(S,v);
    v=0;
    break;
```

Similarly, `;` terminates a rule, and behaves somewhat as a `)`. However, no term-structure with an outermost function symbol is created for rules, because a linked list with pairs is more efficient. That list is needed in reverse textual order, so the pair of the left- and right-hand sides is left on the stack.

```
case ';':
    if (isFUN(v)) v=idx(new(v,0));
    V = Pop(S);
    Push(S, idx(new(V,v)));
    v=0; V=0;
    break;
```

Finally, two situations might occur:

- A term is read (`S==nil`)
  The (possibly coerced) term is returned.
- A TRS is read (`S!=nil`)
  The stack contains the pairs of left- and right-hand sides in reverse order. It is returned (`S` is reset)

```
    case EOF:
    if (S!=nil) {
    v=idx(S);
    S=nil;
    }
    if (isFUN(v)) v=idx(new(v,0));
    return v;
    default:
    error("unexpected character %c (%d)",c,c);
    };
    c = fgetc(in);
    }
}
```

12.1 Scanner / Parser in Tram

The main loop is implemented in the function `sccc`, short for 'scan with character class'. This function implements the `switch` statement of the automaton and has these arguments `sccc(Cc, C, S, V, Stck)`

- `C` is the character currently being considered
- `Cc` is its character class
- `S` is the remainder of the input (string)
- `V` is the current value
- `Stck` is the current stack

Several aspects merit explanation:

- Character class `spec` concerns data-values, which start with `'` or `#`
- Identifiers are scanned by `scsym`; the character class of the first character (`low` or `cap` identifies symbols and variables, respectively)
- The last rule for `sccc` 'catches' all tokens which follow a term: `,`, `;`, `=` and `)`. They are processed by `scaffix` (scan affix).
- `first` is a function (defined in module `Lists.trm`) which returns the first element of a list or string (`first(lst(X,L)) = X; first(str(C,S)) = C;`). The function is partial: `first(eos)` is a normal form. In the context of the scanner, `first(eos)` means: '*we have attempted to read a character when the input was already exhausted*'. That is, `first(eos)` has the same meaning as `EOF` in C. This situation is handled by `scfinalize` (scan finalize)

Function `sccomm` scans and further ignores a comment.

```
scan(str(C,S)) = sccc(cc(C),C,S,null,eol);

sccc(ws,C,S,V,Stck) =
    sccc(cc(first(S)),first(S),rest(S),V,Stck);
sccc(comm,C,S,V,Stck) = sccomm(S,V,Stck);
sccc(spec,C,S,V,Stck) = scspec(C,S,Stck);
```

```
sccc(cap,C,str(B,S),V,Stck) =
   scsym(cc(B),B,S,cap,str(C,eos),Stck);
sccc(low,C,str(B,S),V,Stck) =
   scsym(cc(B),B,S,low,str(C,eos),Stck);
sccc(lpar,C,S,V,Stck)
  = sccc(cc(first(S)),first(S),
          rest(S),null,lst(trm(V,eoa),Stck));
sccc(cc(first(eos)),C,S,V,Stck) = scfinalize(C,S,V,Stck);
sccc(CC,C,S,V,Stck) = scaffix(CC,S,V,Stck);

sccomm(str(#0xA,S),V,Stck) =
   sccc(cc(first(S)),first(S),rest(S),V,Stck);
sccomm(str(C,S),V,Stck) = sccomm(S,V,Stck);
```

Tram does not have a notation for character classes, so all characters are shown explicitly. The function `cc` determines the character class of every valid input character

```
cc('*') = low;  cc('&') = low;  cc('a') = low;  cc('b') = low;
cc('c') = low;  cc('d') = low;  cc('e') = low;  cc('f') = low;
cc('g') = low;  cc('h') = low;  cc('i') = low;  cc('j') = low;
cc('k') = low;  cc('l') = low;  cc('m') = low;  cc('n') = low;
cc('o') = low;  cc('p') = low;  cc('q') = low;  cc('r') = low;
cc('s') = low;  cc('t') = low;  cc('u') = low;  cc('v') = low;
cc('w') = low;  cc('x') = low;  cc('y') = low;  cc('z') = low;

cc('@') = cap;  cc('\mydollar') = cap;  cc('A') = cap;
   cc('B') = cap;
cc('C') = cap;  cc('D') = cap;  cc('E') = cap;  cc('F') = cap;
cc('G') = cap;  cc('H') = cap;  cc('I') = cap;  cc('J') = cap;
cc('K') = cap;  cc('L') = cap;  cc('M') = cap;  cc('N') = cap;
cc('O') = cap;  cc('P') = cap;  cc('Q') = cap;  cc('R') = cap;
cc('S') = cap;  cc('T') = cap;  cc('U') = cap;  cc('V') = cap;
cc('W') = cap;  cc('X') = cap;  cc('Y') = cap;  cc('Z') = cap;

cc('0') = num;  cc('1') = num;  cc('2') = num;  cc('3') = num;
cc('4') = num;  cc('5') = num;  cc('6') = num;  cc('7') = num;
cc('8') = num;  cc('9') = num;

cc('(') = lpar; cc(')') = rpar; cc(',') = sepa;
cc('=') = equ;  cc(';') = sepa; cc(':') = colon; ! cc('.') =
   period;
cc(#0x27) = spec; cc('#') = spec;
cc('!') = comm; cc('"') = quot;

cc(#0x20) = ws;  cc(#0xA) = ws;  cc(#0xD) = ws;  cc(#0x9) =
   ws;
```

Note: this scanner doesn't treat `.` correctly => ToDo.

12.1.1 `scaffix`

Function `scaffix` processes a symbol immediately following a term (i.e., `,`, `=`, `;` and `)`).

An = only occurs after the (possibly coerced) left-hand side of a rule. A partial rule is pushed with only the left-hand side as argument.

Otherwise, the top of the stack is a partial term, and the current value ((possibly coerced) term or variable) is added as the last argument in the top of the stack.

```
scaffix(equ,S,trm(F,As),Stck)
= sclini(equ,S,lst(trm(str('r',str('l',eos)),
                       arg(trm(F,As),eoa)),Stck));
scaffix(equ,S,F,Stck)
= sclini(equ,S,lst(trm(str('r',str('l',eos)),
                       arg(trm(F,eoa),eoa)),Stck));
scaffix(Cc,S,var(V),lst(T,Stck))
= sclini(Cc,S,lst(append(var(V),T),Stck));
scaffix(Cc,S,str(F,Fs),lst(T,Stck))
= sclini(Cc,S,lst(append(trm(str(F,Fs),eoa),T),Stck));
scaffix(Cc,S,trm(F,As),lst(T,Stck))
= sclini(Cc,S,lst(append(trm(F,As),T),Stck));
scaffix(Cc,S,F,lst(T,Stck))
= sclini(Cc,S,lst(append(trm(F,eoa),T),Stck));
```

12.1.2 sclini

Function `sclini` accepts a) or any of (, , = and ;). In the first case (done; the last term is complete), the top-of-stack is popped and set as current value; in the second case (not yet done, the last term is incomplete and remains on the stack to be extended).

```
sclini(rpar,S,lst(T,Stck)) =
   sccc(cc(first(S)),first(S),rest(S),T,Stck);
sclini(Cc,S,Stck) =
   sccc(cc(first(S)),first(S),rest(S),null,Stck);
```

12.1.3 scfinalize and scancollrls

Function `scfinalize` processes EOF. yielding either the list of pairs of left- and right-hand sides, or a term (possibly an extended symbol).

Note that whereas the C implementation needs the rules in reverse order (because thay need to be joined by the `-C` option), the Tram version can at this moment only process a single module. But: rules have been pushed in reverse order, so function `scancollrls` reverses the order of the rules.

```
scfinalize(first(eos),rest(eos),trm(F,As),eol) = trm(F,As);
scfinalize(first(eos),rest(eos),F,eol) = trm(F,eoa);
scfinalize(first(eos),rest(eos),null,Stck) =
   sccollrls(eol,Stck);

sccollrls(Rs,lst(trm(str('r',str('l',eos)),arg(L,arg(R,eoa))),Stck))
 = sccollrls(trm(str('r',str('l',eos)),
                 arg(L,arg(R,arg(Rs,eoa)))),Stck);
sccollrls(V,eol) = V;
```

12.1.4 scsym, scspec, scdec and schx

Function `scsym` parses an id and returns a symbol (string) or a variable.

Function scspec parses a data value. Unlike C, Tram doesn't need the 32-bit version of data, so all data values are represented as strings.

Since Tram doesn't have character classes, auxiliary functions scdec and schx scan decimal and hexadecimal numbers.

```
scsym(low,C,str(B,S),CC,V,Stck) =
    scsym(cc(B),B,S,CC,cat(C,V),Stck);
scsym(cap,C,str(B,S),CC,V,Stck) =
    scsym(cc(B),B,S,CC,cat(C,V),Stck);
scsym(num,C,str(B,S),CC,V,Stck) =
    scsym(cc(B),B,S,CC,cat(C,V),Stck);
scsym(X,C,S,low,V,Stck) = sccc(X,C,S,reverse(V),Stck);
scsym(X,C,S,cap,V,Stck) = sccc(X,C,S,var(reverse(V)),Stck);

scspec('#',str('0',str('x',S)),Stck) = schx(S,eos,Stck);
scspec('#',str('0',S),Stck) = scdec(S,str('0',eos),Stck);
scspec('#',str('-',S),Stck) = scdec(S,str('-',eos),Stck);
scspec('#',S,Stck) = scdec(S,eos,Stck);
scspec(#0x27,str(C,str(#0x27,S)),Stck)
 = sccc(cc(first(S)),first(S),rest(S),C,Stck);

scdec(str('0',S),V,Stck) = scdec(S,str('0',V),Stck);
scdec(str('1',S),V,Stck) = scdec(S,str('1',V),Stck);
scdec(str('2',S),V,Stck) = scdec(S,str('2',V),Stck);
scdec(str('3',S),V,Stck) = scdec(S,str('3',V),Stck);
scdec(str('4',S),V,Stck) = scdec(S,str('4',V),Stck);
scdec(str('5',S),V,Stck) = scdec(S,str('5',V),Stck);
scdec(str('6',S),V,Stck) = scdec(S,str('6',V),Stck);
scdec(str('7',S),V,Stck) = scdec(S,str('7',V),Stck);
scdec(str('8',S),V,Stck) = scdec(S,str('8',V),Stck);
scdec(str('9',S),V,Stck) = scdec(S,str('9',V),Stck);
scdec(S,V,Stck) =
    sccc(cc(first(S)),first(S),rest(S),reverse(V),Stck);

schx(str('a',S),V,Stck) = schx(S,str('A',V),Stck);
schx(str('b',S),V,Stck) = schx(S,str('B',V),Stck);
schx(str('c',S),V,Stck) = schx(S,str('C',V),Stck);
schx(str('d',S),V,Stck) = schx(S,str('D',V),Stck);
schx(str('e',S),V,Stck) = schx(S,str('E',V),Stck);
schx(str('f',S),V,Stck) = schx(S,str('F',V),Stck);
schx(str('A',S),V,Stck) = schx(S,str('A',V),Stck);
schx(str('B',S),V,Stck) = schx(S,str('B',V),Stck);
schx(str('C',S),V,Stck) = schx(S,str('C',V),Stck);
schx(str('D',S),V,Stck) = schx(S,str('D',V),Stck);
schx(str('E',S),V,Stck) = schx(S,str('E',V),Stck);
schx(str('F',S),V,Stck) = schx(S,str('F',V),Stck);
schx(str('0',S),V,Stck) = schx(S,str('0',V),Stck);
schx(str('1',S),V,Stck) = schx(S,str('1',V),Stck);
schx(str('2',S),V,Stck) = schx(S,str('2',V),Stck);
schx(str('3',S),V,Stck) = schx(S,str('3',V),Stck);
```

```
schx(str('4',S),V,Stck) = schx(S,str('4',V),Stck);
schx(str('5',S),V,Stck) = schx(S,str('5',V),Stck);
schx(str('6',S),V,Stck) = schx(S,str('6',V),Stck);
schx(str('7',S),V,Stck) = schx(S,str('7',V),Stck);
schx(str('8',S),V,Stck) = schx(S,str('8',V),Stck);
schx(str('9',S),V,Stck) = schx(S,str('9',V),Stck);
schx(S,V,Stck) =
   sccc(cc(first(S)),first(S),rest(S),reverse(V),Stck);
```

13. Converting Tram To C

Section Tram – A Meta-Interpreter describes a rewrite engine on meta-terms. That specification is used as the basis for the C implementation in TRAM.1.

13.1 Preamble

13.1.1 States

A big difference between term rewriting and the C programmers' model is the limited availability of recursion in C. Section Converting a Scanner from C to Tram uses an explicit stack to avoid recursion in C, in a way transforming a recursive descent parser to a push-down automaton.

But the simple parser has only a single state in which recursion is initiated (after reading a `(`), and only a single state after which it is returned from (after `)`). In the rewrite engine, more states exist. Consider the first two rules in the meta-interpreter:

```
burewr(trm(F,As),Rs) = toprewr(trm(F,burewr(As,Rs)),Rs);
burewr(arg(A,As),Rs) = arg(burewr(A,Rs),burewr(As,Rs));
```

In Rule 1, after `burewr` returns a result, `toprewr` must be called. That pattern, *after this do that,* suggests a state, which is either maintained explicitly or returned by `burewr` (in this case). In Rule 2, there are two recursive calls to `burewr`, so these are (at least) two additional states.

13.1.2 Nodes

A second difference between the meta-terms and the rewrite engine has been discussed in Section Nodes: TRAM represents terms as nodes (structures with two fields: `car` and `cdr`) (ignoring memory management for the moment). This means

- there is no distinction (in TRAM) between `trm(...)` and `arg(...)`. A `trm(...)` head-node is recognized by the fact that the `car` is a symbol
- a variable in the meta-term representation (`var(name)`) is a base value in TRAM (a 32-bit value with least significant byte binary 00000011)

13.1.3 Basics

The entire implementation of TRAM.1 is discussed in this section, so here we only mention a few primitives used in the rewrite engine.

- predicates to discern values

```
#define isREF(t) (((t)&1)==0)
#define isNREF(t) (((t)&1)==1)
#define isVAR(t) (((t)&0xff)==3)
#define isFUN(t) (((t)&3==3)&&((t)&0xff)>3)
```

- Conversion pointer <==> reference value

```
#define ref(t) (mem+(t)/2)
#define idx(r) ((r-mem)*2)
```

- Node structure and function to create a new node

```
typedef struct _node {
    tval car, cdr, nxt;
} node;
typedef node *ref;

ref new(tval x,tval y);
```

- Stack manipulation

```
#define Push(X,a) X=new(a,idx(X));
#define Pop(X) X->car; X=ref(X->cdr);
#define PopRef(X) ref(X->car); X=ref(X->cdr);
```

13.2 TRAM.1's Rewrite Engine

The meta-interpreter discussed in Section Term Rewriting.

The rewrite engine is a push-down automaton where the main loop branches based on the current state. The initial state is BURED to invoke bottom-up reduction given a term in register T using 'program' (TRS) in register P.

```
ref reduce (/*tval T, ref P*/) {
    S = nil;
    int state=BURED;
    tval f, t, pat;
    ref p, sub;
    Push(S,asDTA(ALLDONE));
    for (;;) {
    if (Dbg >= DCycles) fprintf(stdout, "%s ", prstates[state
   - 1000]);
    switch (state) { // T is subject term, V is result
```

13.2.1 burewr

```
1   burewr(trm(F,As),Rs) = toprewr(trm(F,burewr(As,Rs)),Rs);
2   burewr(arg(A,As),Rs) = arg(burewr(A,Rs),burewr(As,Rs));
3   burewr(eoa,Rs) = eoa;
```

Note that in TRAM, there is no structural distinction between `trm(...)` and `arg(...)`.

Rules 1-3 of the meta-interpreter define bottom-up rewriting. An imperative description of rules 1-3 is:

- Rules 1-2 push down `burewr` through an entire tree, such that `burewr` is applied to each node in that tree, which replaces that node with its `burewr` image
- Rule 3 states that an empty tree isn't changed by `burewr`;
- Rule 1 states that once a proper sub-term has been normalized, `topred` should be applied.

Four states can be distinguished:

- `BURED`, the initial state
- `BUDONECDR`. TRAM implements the right-most innermost strategy, so the sub-term `burewr(As, Rs)` is reduced first (in rules 1 and 2). The result will be captured by the state at which the `cdr` of the `arg(...)` or `trm(...)` node has been handled (`BUDONECDR` for bottom-up, done cdr)
- `BUDONEBOTH` is self-described. In the case of Rule 2, `burewr` has been applied to the entire tree; in the case of Rule 1, the next state should now be visited:
- `TOPRED`, try to reduce a term at root level

Lines

- 2-4: A base value or the null pointer is irreducible (as in Rule 3). Pop the next state
- 7-9: Otherwise, save the `car`, push the subsequent state, and continue with the `cdr`
- 12-15: Pop the `car`, push the normalized `cdr` and the subsequent state, and recurse
- 18-21: If the car is a node, this is an `arg(...)` node; create the new node (as in Rule 2)
- 24-26: Otherwise, this is a normalized `trm(...)`. Apply `TOPRED`

```
1  case BURED: //T is term
2      if (isNREF(T) || T == 0) {
3          V = T;
4          state = PopDTA(S);
5          break;
6      }
7      Push(S, ref(T)->car);
8      Push(S, asDTA(BUDONECDR));
9      T = ref(T)->cdr;
10     break;
11 case BUDONECDR: //V is cdr, tos is car-to-do
12     T = Pop(S);
13     Push(S, V);
14     Push(S, asDTA(BUDONEBOTH));
15     state = BURED;
16     break;
17 case BUDONEBOTH: //(V is car), tos is cdr
18     if (isREF(V)) {
19         T = Pop(S);
20         V = idx(new(V, T));
21         state = PopDTA(S);
22         break;
```

```
23      }
24      f = V;
25      T = Pop(S);
26      state = TOPRED;
27      break;
```

13.2.2 toprewr

```
4    toprewr(T,Rs) = toprewrRule(T,Rs,Rs);
5    toprewrRule(T,trm("rl",arg(L,arg(R,arg(Rs,eoa)))),TRS)
     = match(L,T,eol,eoa,eoa,R,T,Rs,TRS);
6    toprewrRule(T,trm("eor",eoa),Rs) = T;
. . .
10   match(eoa,eoa,E,eoa,eoa,R,T,Rs,TRS) =  inst(R,E,TRS);
. . .
13   matchq(X,Fs,Gs,E,As,Bs,R,T,Rs,TRS) =
    toprewrRule(T,Rs,TRS);
```

Function `toprewr` sets up for `toprewrRule` to attempt each rule in the program/TRS in function `match` . If this succeeds, the right-hand side is instantiated (Rule 10); if it fails, function `match` should attempt the next alternative (Rule 13).

Five states can be distinguished:

- `TOPRED`, as mentioned, sets up the loop at root level
- `FORRULES`, a 'for' loop over all rules
- `MATCH`, the state in which a term is matched
- `MATCHDONE`: matching can fail deep in the recursion. The token `MATCHDONE` is pushed to be able to clear the stack when matching fails, or it is encountered on the stack when all work-to-do has been done
- `BUILD`. There is a significant difference between the meta-interpreter and the C implementation if matching fails. In the meta-interpreter, the term being reduced has already been built and can be passed on as-is (term T in Rule 13). In the C implementation the term hasn't yet been built and exists as a separate symbol `F` and arguments `T` (See Lines 24-25 in the C code above).

```
1  case TOPRED: //(f is fun,T = args)
2      if (Dbg >= DStepDump) pval(f, 1);
3      p = P;
4      t = T;
5      state = FORRULES;
6      if (Dbg >= DSteps) Drulei = 0;
7      break;
8  case FORRULES: //(f,t,p, T, P)
9      if (p == nil) { // eor
10         state = BUILD;
11         break;
12     }
13     if (Dbg >= DSteps) Drulei++;
14     if (ref(ref(p->car)->car)->car != f) {
15         p = ref(p->cdr);
16         break;
```

```
17      }
18      sub = nil;
19      Push(S, asDTA(MATCHDONE));
20      state = MATCH;
21      pat = ref(ref(p->car)->car)->cdr;
22      break;
23  case MATCH: //(t,pat,sub,p, T, P)
```

Lines

- 3-5: set up loop
- 9-10: no match; build the term
- 14-15: the outermost function symbol of the subject term and that of the left-hand side of the rule **differ**; the next rule is attempted
- 18-21: the outermost function symbol of the subject term and that of the left-hand side of the rule **are the same**; matching is set up with an as-yet empty substitution

13.2.3 `match`

The arguments in `match(X,ST,E,As,Bs,R,T,Rs,TRS)` are:

- `X`: the current pattern (sub-term of the left-hand side of the rule)
- `ST`: the current sub-term being matched
- `E`: the substitution in the form of `tab(<variable-name>, <value>, <rest-of-substitution>)`. No check is done to see if the variable was already defined. That would constitute a non-linear TRS
- `As`, `Bs`: the sub-terms of the subject term and pattern that still need to be matched. Note that constructor `arg(...)` is used to string together sub-terms that need to be inspected. In this sense, `As` and `Bs` could also be described as stacks of work-yet-to-be-done
- `R`: the right-hand side of the current rule
- `T`: the entire subject-term, to be used if matching fails
- `Rs`: the remaining rules, to be used if matching fails
- `TRS`: the entire rewrite system, to be used after the current subject term is reduced

```
7    match(var(N),ST,E,As,Bs,R,T,Rs,TRS)
     = match(As,Bs,tab(N,ST,E),eoa,eoa,R,T,Rs,TRS);
8    match(trm(F,Fs),trm(G,Gs),E,As,Bs,R,T,Rs,TRS)
     = matchq(eq(F,G),Fs,Gs,E,As,Bs,R,T,Rs,TRS);
9    match(arg(P,Ps),arg(Q,Qs),E,As,Bs,R,T,Rs,TRS)
     = match(P,Q,E,arg(Ps,As),arg(Qs,Bs),R,T,Rs,TRS);
10   match(eoa,eoa,E,eoa,eoa,R,T,Rs,TRS) =  inst(R,E,TRS);
11   match(eoa,eoa,E,arg(A,As),arg(B,Bs),R,T,Rs,TRS)
     = match(A,B,E,As,Bs,R,T,Rs,TRS);
12   matchq(true,Fs,Gs,E,As,Bs,R,T,Rs,TRS)
     = match(Fs,Gs,E,As,Bs,R,T,Rs,TRS);
13   matchq(X,Fs,Gs,E,As,Bs,R,T,Rs,TRS) =
    toprewrRule(T,Rs,TRS);
```

The logic is straightforward:

- Rule 7-11 match different forms of a pattern
- Rule 7 matches a variable, which always succeeds, and which results in substitution being extended
- Rules 8-9 match term-trees. Note that matching a `trm(...)` against an `arg(...)` would signify a syntactical error in the subject term or TRS

- Rule 8: if the sub-term and pattern have the shape of a symbol applied to arguments, compare the symbols
- Rule 9: continue matching in the first sub-term, and push the remainder to be done
- Rule 10:If the sub-term has been checked, and if the work yet to be done is exhausted, all matches have succeeded, and the rule can be applied
- Rule 11: otherwise, the next sub-term to be done is matched against
- Rules 12-13: auxiliary function `matchq` proceeds or fails, according to (in) equality of symbols

Five states can be distinguished:

- `MATCH`, `MATCHDONE`: as mentioned
- `fail` is not a state but a label to be jumped to when matching fails
- `MATCHDONECAR`: the `car` has been verified; continue with the `cdr`
- `INST`: match succeeded; instantiate the right-hand side

```
case MATCH: //(t,pat,sub,p, T, P)
    if (isREF(pat) && pat != 0) { // compound
        if (isNREF(t)) goto fail;
        Push(S, ref(t)->cdr);
        Push(S, ref(pat)->cdr);
        Push(S, asDTA(MATCHDONECAR));
        pat = ref(pat)->car;
        t = ref(t)->car;
        break;
    }
    if (isVAR(pat)) {
        sub = new(idx(new(pat, t)), idx(sub));
        state = PopDTA(S);
        break;
    }
    if (pat == t) {
        state = PopDTA(S);
        break;
    }
    //fallthrough intended
fail:
    p = ref(p->cdr);
    t = T;
    do {
        state = PopDTA(S);
    } while (state != MATCHDONE);
    state = FORRULES;
    break;
case MATCHDONECAR://(sub,p) tos is pat.cdr, 2nd is trm.cdr
    pat = Pop(S);
    t = Pop(S);
    state = MATCH;
    break;
case MATCHDONE://(sub,p)
    pat = ref(p->car)->cdr; //rhs
    state = INST;
```

```
37      if (Dbg >= DSteps) Drewrcnt++;
38      if (Dbg >= DSteps) printf("Rule %d, Steps %d\n", Drulei,
    Drewrcnt);
39      break;
40  case INST: //(pat,sub)
```

Lines

- 3: if the pattern is a node, and the sub-term is not, matching fails
- 4-8: otherwise, push the `cdr`'s and continue with the `car` (i.e., recurse)
- 11-13: add a variable and value to the substitution
- 16-17: otherwise, pattern and term are base values. They only match if they are equal
- 22-27: failure to match sets the program to the next rule; sets the current term to the initial term; clears the stack; and continues the loop
- 30-32: pop the cdr and continue matching
- 35-36: get and instantiate the right-hand side

13.2.4 `inst`

Function `inst` is trivial; no additional states ensue.

```
14  inst(var(N),E,TRS) = get(N,E);
15  inst(trm(F,As),E,TRS) =
    toprewr(trm(F,inst(As,E,TRS)),TRS);
16  inst(eoa,E,TRS) = eoa;
17  inst(arg(A,As),E,TRS) = arg(inst(A,E,TRS),inst(As,E,TRS));
18  get(N,tab(M,T,E)) = getq(eq(N,M),N,T,E);
19  getq(ok,N,T,E) = T;
20  getq(X,N,T,E) = get(N,E);
```

- Rule 14, 18-20: for every variable, get its associated value
- Rules 15-17: recursively instantiate all sub-terms of the right-hand side
- Rule 15: for every compound subterm, reduce it at the top level. Since an innermost strategy is followed, the values of variables have already been normalized

```
1  case INST: //(pat,sub)
2      if (isREF(pat) && pat != 0) {
3          Push(S, ref(pat)->cdr);
4          pat = ref(pat)->car;
5          Push(S, asDTA(INSTDONECAR));
6          break;
7      }
8      if (isVAR(pat)) {// use p as tmp in sub
9          p = sub;
10         while (ref(p->car)->car != pat) {
11             p = ref(p->cdr);
12         }
13         V = ref(p->car)->cdr;
14         state = PopDTA(S);
15         break;
16     }
17     V = pat;
18     state = PopDTA(S);
```

```
19      break;
20  case INSTDONECAR: // V is car
21      pat = Pop(S);
22      Push(S, V);
23      Push(S, asDTA(INSTDONEBOTH));
24      state = INST;
25      break;
26  case INSTDONEBOTH: // V is cdr, ToS = car
27      t = Pop(S);
28      if (isFUN(t)) {
29          f = t;
30          T = V;
31          Push(S, pat);
32          Push(S, idx(sub));
33          Push(S, asDTA(INSTCONT));
34          state = TOPRED;
35          break;
36      }
37      V = idx(new(t, V));
38      state = PopDTA(S);
39      break;
40  case INSTCONT: {// V is car
41      sub = PopRef(S);
42      pat = Pop(S);
43      state = PopDTA(S);
44      break;
45      }
```

Lines

- 2-5: for a compound, push the `cdr` and continue with the `car`
- 9-14: for a variable, fetch the corresponding value. This implements Rules 14, 18-20
- 17-18: done; continue
- 21-24: save the `car`, continue with the `cdr`
- 27-28: is this a `trm(...)` node or an `arg(...)` node?
- 29-34: topreduce entire `trm(...)`. Note that the current state of instantiation is saved in lines 31-33. Executing `TOPRED` is deep recursion that may require many rewrite steps
- 37-38: otherwise, build the `arg(...)` node and continue
- 41-43: a saved instantiation is continued after intermediate `TOPRED` of proper sub-term of the right-hand side

13.2.5 ALLDONE

The final segment is trivial.

- 2-3: as mentioned, build a node if no rule is applicable
- 6-7: if all seems well, return the normal form

```
1  case BUILD:
2      V = idx(new(f, T));
3      state = PopDTA(S);
4      break;
5  case ALLDONE:
```

```
    if (S != nil) error("Stack should be empty!");
    return ref(V);
default: error("Bad state!");
}
```

14. Patterns: Encoding-Checker from Scratch

In Tram.1, only the first five characters in a function name are significant (plus the fact whether they are longer or not; four in variables). But TRAM.1 doesn't check this: if two distinct function names are used, the first five characters of which are equal, then they are encoded to the same value.

To avoid bugs, a checker is needed, which checks if this occurs in an input TRS. Program `EncChecker` checks all variables and functions and warns if unequal identifiers will be encoded the same.

`EncChecker` uses the scanner/parser we have described in Section Converting from C to Tram (Scanner/Parser) and auxiliary modules `Strings`, `Lists`. All code can be found in the TRAM.1 Github repository.

Our purpose in describing `EncChecker` is not only to document that tool but mostly to discuss a few term rewriting programming patterns.

Given a TRS, function `encchk` should indicate which functions or variables are used which differ and yet are encoded the same. We do not focus on generating nice user-oriented output. Instead, `encchk` will output a list of all identifiers used in a program and will place unequal identifiers that lead to the same value (encoding) next to each other, preceded by `$$` (which is not a legal identifier and therefore easily recognized).

That is, given a text file, the term `encchk(scan(%1))` would produce such a list.

14.1 Pattern: Initializing Local Variables

Tram has no local variables, and yet, some functions need auxiliary data structures, which must be initialized. Inside module `EncChecker`, a list of all identifiers must be maintained, which is initialized to the empty list.

```
encchk(T) = ck(T,eol);
```

The meta-term representation produced by `scan` generates four types:

1. `trm(F,As)` for symbol `F` and argument list `As`
2. `arg(A,As)` for argument (term) `A` and argument list `As`
3. `eoa` the empty argument list

4. `var(N)` variable with name `N`

14.2 Pattern: Function Name or Extra Parameter

The checker could maintain two lists: one for functions and one for variables, but for simplicity, a single list is used. However, encoding rules for functions and variables differ, so processing `F` in Case 1 above and `N` in Case 4 must be distinguished.

The **function name pattern** might distinguish these cases as follows:

```
ck(trm(F,As),L) = ck(As,mpqf(F,L));
...
ck(var(N),L) = mpqv(N,L);
```

But the rules for `mpqf` and `mpqv` are likely to be very similar.

The **extra parameter pattern** encodes the distinction in a data value. In this case the character class of the first character in the identifier is used: `low` for functions, and `cap` for variables:

```
ck(trm(F,As),L) = ck(As,mpq(low,F,L));
...
ck(var(N),L) = mpq(cap,N,L);
```

The extra parameter pattern is used if further processing is similar, possibly diverging further along the way. We have seen this pattern being used in the function `sccc` in the scanner. The function name pattern is more common and is also used in function `sccc`.

The purpose of function `ck` is to visit all identifiers in a TRS and process them into the growing list of identifiers:

```
ck(trm(F,As),L) = ck(As,mpq(low,F,L));
ck(arg(A,As),L) = ck(As,ck(A,L));
ck(eoa,L) = L;
ck(var(N),L) = mpq(cap,N,L);
```

Function `mpq` (mnemonic: *map-query*) maps the subject identifier to the entire list of stored identifiers, querying each if it leads to the same encoding. It creates a new list if there are no further identifiers to compare or passes the subject's character class and the character class of the first stored identifier to the auxiliary function `mpqc`.

```
mpq(Cc,Id,eol) = lst(Id,eol);
mpq(Cc,Id1,lst(Id2,L)) = mpqc(Cc,cc(first(Id2)),Id1,Id2,L);
```

14.3 Pattern: Have Your Cake

Ordinarily, sub-terms of arguments are accessed by matching the argument against a pattern. Function `mpq` could have been defined by

```
mpq(Cc,Id,eol) = lst(Id,eol);
mpq(Cc,Id1,lst(str(C,S),L)) = mpqc(Cc,cc(C),Id1,str(C,S),L);
```

Where the earlier definition passes `Id2`, the latter definition passes `str(C, S)`, which is indeed the same argument. But in an implementation, a new term will then be built, which happens to be an equal term. This might be a small concern, but the earlier definition (to me) seems clearer.

The pattern leaves an argument as-is and uses an accessor (`first`, in this case) to access a sub-term. The pattern might be called *'have your cake and eat it'*. Note that there is an implementational

disadvantage here as well: an additional reduction to reduce `first`. In most cases, clarity should prevail.

The next function, `mpqc`, should distinguish two situations: the subject and the first identifier in the list have the same type or not. If they are the same type, `four` and `five` are used to identify the number of characters that should coincide. We could also use values #4 and #5, but since no built-in operators exist for data values, they offer little added value.

```
mpqc(low,low,Id1,Id2,L)
  = eqnc(five,eq(first(Id1),first(Id2)),rest(Id1),rest(Id2),
         low,Id1,Id2,L);
mpqc(cap,cap,Id1,Id2,L)
  = eqnc(four,eq(first(Id1),first(Id2)),rest(Id1),rest(Id2),
         cap,Id1,Id2,L);
mpqc(Cc1,Cc2,Id1,Id2,L) = lst(Id2,mpq(Cc1,Id1,L));
```

Note that the 'have-your-cake-pattern' is used again. Without it the specification would have been almost as clear:

```
mpqc(low,low,str(C1,S1),str(C2,S2),L)
  = eqnc(five,eq(C2,C2),S1,S2,low,str(C1,S1),str(C2,S2),L);
mpqc(cap,cap,str(C1,S1),str(C2,S2),L)
  = eqnc(four,eq(C2,C2),S1,S2,cap,str(C1,S1),str(C2,S2),L);
mpqc(Cc1,Cc2,Id1,Id2,L) = lst(Id2,mpq(Cc1,Id1,L));
```

Finally, the heart of the checker (the function `eqnc`, for *equal encoding*) should distinguish these situations: 1. the identifiers have been entirely compared and are equal. The first identifier can be ignored 2. the characters relevant for the encoding are equal. Now the remaining characters must be compared 3. all characters are equal so far, but more characters must be compared within the encoding 4. an inequality is found beyond the length of the encoding: two different identifiers will be encoded the same 5. an inequality is found within the encoding length. These are simply distinct identifiers

```
1 eqnc(N,ok,eos,eos,Cc,Id1,Id2,L) = lst(Id2,L);
2 eqnc(zero,ok,str(C1,S2),str(C2,S2),Cc,Id1,Id2,L)
  = eqnc(zero,eq(C1,C2),S1,S2,Cc,Id1,Id2,L);
3 eqnc(N,ok,str(C1,S2),str(C2,S2),Cc,Id1,Id2,L)
  = eqnc(prev(N),eq(C1,C2),S1,S2,Cc,Id1,Id2,L);
4 eqnc(zero,P,Q,R,Cc,Id1,Id2,L)
  =
   lst(str('\mydollar',str('\mydollar',eos)),lst(Id1,lst(Id2,L)));
5 eqnc(N,P,Q,R,Cc,Id1,Id2,L) = lst(Id2,mpq(Cc,Id1,L));
```

Finally, auxiliary function `prev` is defined:

```
1 prev(five) = four; prev(four) = three; prev(three) = two;
   prev(two) = one; prev(one) = zero;
```

In the repository at TRAM.1 Github repository a single rule is added to file `EncChecker` to test it on itself:

```
1 badfn1(Badv1,Badv2) = badfn2;
```

Both functions and variables should flag an error. The output of `encchk(scan(%1))` on `EncChecker` itself:

```
lst("rl",lst("encchk",lst("T",lst("ck",lst("eol",lst("trm",
lst("As",lst("mpq",lst("low",lst("arg",lst("eoa",lst("var",
lst("cap",lst("Cc",lst("Id",lst("lst",lst("Id1",lst("Id2",
lst("mpqc",lst("cc",lst("first",lst("str",lst("C1",lst("C2",
lst("eqnc",lst("five",lst("eq",lst("four",lst("Cc1",
lst("Cc2",lst("eos",lst("zero",lst("Idr1",lst("Idr2",
lst("rest",lst("prev",lst(#0x24,lst(#0x24,lst("three",
lst("two",lst("one",lst("\mydollar\mydollar",lst("badfn2",lst("badfn1",
lst("\mydollar\mydollar",lst("Badv2",lst("Badv1",lst("eor",
eol))))))
```

Conclusions

15. Conclusions

MinimalMagic.blog/trs was created with several purposes in mind:

- As an informal though complete and entirely self-contained introduction to term rewriting systems.
- As an illustration of the principle of *minimal magic*: develop complex systems out of (trivial) first principles
- As documentation for Tram: the *Term Rewriting Abstract Machine*
- As a guide to writing term rewriting systems from scratch or given a C implementation, and to writing a C implementation given a term rewriting system

Chapter 5 offers such an introduction, which (informally) sketches set theory and defines the theory of term rewriting systems based thereon. Chapter 6 defines the meta-notation of terms, which is the key to compilers and higher-order transformation (in Tram's first-order theory). But Chapter 6 also (formally) defines the right-most innermost term rewriting, finalizing a correct and complete characterization of term rewriting.

Part II chapters 7-11, offer documentation for Tram, and in particular for the implementation TRAM.1, by explaining key algorithms and annotating key components.

Part III, chapters 12-14 illustrate the programming process by discussing how to create term rewriting systems given a C implementation of an algorithm, and the reverse: creating a C implementation given an abstract term rewriting specification. Also, a few design patterns specific to term rewriting are discussed.

Postamble

16. Acknowledgements

Legrand Orange Book Template, licensed under **CC BY-NC-SA 4.0**.

Special thanks to my friend Jamie for reviewing and sharing.

Photos from Pexels:

- Цвета Тишины
- Aaron Burden
- Aarti Vijay
- Anderson Guerra
- Andy Vu
- Björn Austmar Þórsson
- Fiona Art
- Francesco Ungaro
- Giovanni Figueroa
- Harrison Candlin
- Humphrey Muleba
- Isabella Mariana
- Jakob Rosen
- James Wheeler
- Kristina Chuprina
- Longxiang Qian
- Luis del Río
- Max Ravier
- Oday Hazeem
- Pixabay
- Pratik Gupta
- Roberto Nickson
- Rodrigo Souza
- Santiago Pagnotta
- Scott Webb

- Serkan Atay
- Sharon McCutcheon
- Silvia
- Simon Berger
- Sofía Rabassa
- Stefan Stefancik
- Steve Johnson
- Toa Heftiba Şinca
- Victor Dompablo
- Will Mu

17. References

- Baader, F., & Nipkow, T. (2009).
  Term rewriting and all that. Cambridge Univ. Press.
- Fokkink, W., Kamperman, J., & Walters, P. (1998).
  Within ARM's reach. *ACM Transactions on Programming Languages and Systems*, *20*(3), 679–706.
  `https://doi.org/10.1145/291889.291903`
- Fokkink, W., Kamperman, J., & Walters, P. (2000).
  Lazy rewriting on eager machinery. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, *22*(1), 45–86.
  `https://doi.org/10.1145/345099.345102`
- Hertz, M., & Berger, E. D. (2005).
  Quantifying the performance of garbage collection vs. explicit memory management. *ACM SIGPLAN Notices*, *40*(10), 313.
  `https://doi.org/10.1145/1103845.1094836`
- Kamperman, J., & Walters, P. (1995).
  Lazy rewriting and eager machinery. *Rewriting Techniques and Applications*, *vol. 914*, 147–162.
  `https://doi.org/10.1007/3-540-59200-8_54`
- Ohlebusch, E. (2002).
  Advanced topics in term rewriting. Springer.
- Walters, P. (1991).
  A Complete Term Rewriting System for Decimal Integer Arithmetic. Centre for Mathematics & Computer Science.
  `https://www.researchgate.net/publication/2450227_A_Complete_Term_Rewriting_System_for_Decimal_Integer_Arithmetic`
- Walters, P., & Kamperman, J. (1993).
  ARM abstract rewriting machine. Centre for Mathematics & Computer Science.
  `https://www.researchgate.net/publication/2432082_ARM_abstract_rewriting_`

machine, Report CS-R9330

- Walters, P., & Kamperman, J. (1995a).
  A model for I/O in equational languages with don't care non-determinism. *Lecture Notes in Computer Science*, *1130*, pp 522-535.
  https://link.springer.com/chapter/10.1007/3-540-61629-2_61, Springer
- Walters, P., & Kamperman, J. (1995b).
  Minimal Term Rewriting Systems. *Lecture Notes in Computer Science*, *1130*, pp 274-290.
  https://link.springer.com/chapter/10.1007/3-540-61629-2_48, Springer
- Walters, P., & Kamperman, J. (1996).
  Simulating TRSs by minimal TRSs: a simple, efficient, and correct compilation technique.
  Centre for Mathematics & Computer Science.
  https://www.researchgate.net/publication/2237614_Simulating_TRSs_by_Minimal_TRSs_a_Simple_Efficient_and_Correct_Compilation_Technique, Report CS-R9605
- Walters, P., & Zantema, H. (1995).
  Rewrite systems for integer arithmetic. *Lecture Notes in Computer Science*, *vol. 914*, pp. 324-338.
  https://doi.org/https://doi.org/10.1007/3-540-59200-8_67, In: Hsiang J. (eds) Rewriting Techniques and Applications. RTA 1995.